

Breaking the Simplification Bottleneck in Amortized Neural Symbolic Regression

Paul Saegert¹ Ullrich Köthe¹

Abstract

Symbolic regression (SR) aims to discover interpretable analytical expressions that accurately describe observed data. Amortized SR promises to be much more efficient than the predominant genetic programming SR methods, but currently struggles to scale to realistic scientific complexity. We find that a key obstacle is the lack of a fast reduction of equivalent expressions to a concise normalized form. Amortized SR has addressed this with general-purpose Computer Algebra Systems (CAS) like SymPy, but the high computational cost severely limits training and inference speed. We propose **SimpliPy**, a rule-based simplification engine achieving a 100-fold speed-up over SymPy at comparable quality. This enables substantial improvements in amortized SR, including scalability to much larger training sets, more efficient use of the per-expression token budget, and systematic training set decontamination with respect to equivalent test expressions. We demonstrate these advantages in our **Flash-ANSR** framework, which achieves much better accuracy than amortized baselines (NeSymReS, E2E) on the Fast-SRB benchmark. Moreover, it performs on par with state-of-the-art direct optimization (PySR) while recovering more concise rather than more complex expressions with increasing inference budget.

1. Introduction

Symbolic regression (SR) occupies a unique position in scientific machine learning by enabling the discovery of interpretable, closed-form laws from observational data (Schmidt & Lipson, 2009). Unlike standard deep learning, SR seeks the analytical expressions that govern the data-generating

process. Traditionally, this is framed as a combinatorial optimization problem solved via genetic programming (GP) (Koza, 1994; Cranmer, 2023). While GP remains the gold standard for precision, it treats every dataset as a *tabula rasa* search instance, failing to transfer structural knowledge between tasks.

Pioneered by Lample & Charton (2020); Biggio et al. (2021), amortized SR solves this by learning the posterior $p(\text{expression} | \text{data})$ over millions of examples. It aims to shift the computational burden to a one-time pre-training phase, encoding the prior of a simulation-based-inference simulator (SBI; Cranmer et al., 2020) into the model. Meanwhile, neural scaling laws dictate that robust generalization requires data that is vast, diverse (Kaplan et al., 2020), and high-quality (Lample & Charton, 2020; Lee et al., 2022; Gunasekar et al., 2023). This presents a fundamental challenge: randomly generated mathematical expressions are rife with redundancies (e.g., $x + x$ vs. $2x$) that must be *simplified* to ensure high-quality, normalized training targets.

Current state-of-the-art approaches face a dilemma regarding this simplification step. Standard Computer Algebra Systems (CAS) like SymPy (Meurer et al., 2017) rely on heavy object-oriented parsing and tree traversals. When integrated into training loops (Bendinelli et al., 2023; Yu et al., 2025), this creates a *simplification bottleneck*, where data generation becomes orders of magnitude slower than gradient updates. To maintain feasible training times, many compromise on the amount or diversity of the training data, or abandon simplification entirely (Kamienny et al., 2022).

We introduce **SIMPLIPY**¹, an engine that reduces symbolic simplification to fast pattern matching, achieving speedups of up to 100× over SymPy at comparable quality. This breakthrough enables **FLASH-ANSR**² (Figure 1), a framework for Transformer-based training on a continuously generated stream of high-quality expressions. Freed from the common CAS bottleneck, FLASH-ANSR can be scaled to learn to approximate the posterior over a much broader distribution of mathematical functions and data.

Contributions:

1. We identify the simplification bottleneck as a key ob-

¹Computer Vision and Learning Lab, Heidelberg University, Germany. Correspondence to: Paul Saegert <bc226@uni-heidelberg.de>, Ullrich Köthe <ullrich.koethe@iwr.uni-heidelberg.de>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

¹<https://github.com/psaegert/simplipy>

²<https://github.com/psaegert/flash-ansr>

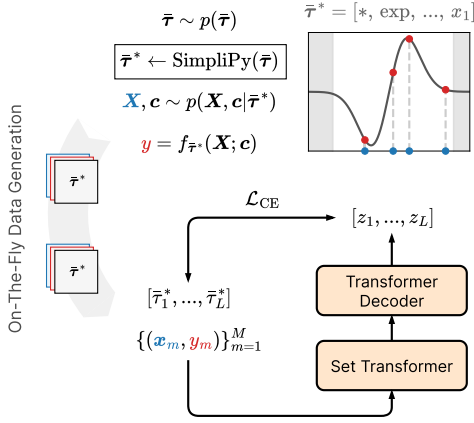


Figure 1. The FLASH-ANSR training pipeline. Following the established standard encoder-decoder paradigm, our framework integrates **SimpliPy** (top center) into the loop for synchronous simplification of on-the-fly generated training expressions.

stacle to scaling the training and inference of amortized SR and introduce SIMPLIPY, a hash-based simplification engine to resolve it.

2. We present FLASH-ANSR, which leverages SIMPLIPY to train on 512M on-the-fly generated and simplified data-expression pairs, scaling to higher-dimensional inputs and broader operator sets than prior work.
3. We demonstrate that FLASH-ANSR dominates the inference-time-recovery-rate Pareto frontier against both static (NeSymReS) and unsimplified (E2E) baselines, and matches state-of-the-art GP methods (PySR) with superior inference-time-parsimony scaling.
4. Addressing the pervasive lack of rigor regarding data leakage and evaluation standards in the field, we implement strict *symbolic and numeric decontamination* of training data, ensuring our results reflect actual generalization. We follow a rigorous *test-time compute* evaluation protocol, exposing the inference time budget trade-offs often obscured in prior literature.

Conflict of Interest Disclosure. The authors declare no conflicts of interest.

2. Problem Formulation

Given a dataset $\mathcal{D} = \{(\mathbf{x}_m \in \mathbb{R}^D, y_m \in \mathbb{R})\}_{m=1}^M$, symbolic regression (SR) seeks an analytical function $f_{\tau}(\cdot; \mathbf{c}): \mathbb{R}^D \rightarrow \mathbb{R}$ that explains the data, defined by a mathematical expression $\tau = [\tau_1, \dots, \tau_L]$ represented as a sequence containing input variables $\mathcal{V} = \{x_1, \dots, x_D\}$, operators $\mathcal{O}$ (e.g. $\{+, *, \sin, \dots\}$), and constants $\mathbf{c} \in \mathbb{R}^K$. The space of expressions $\mathcal{E}$ is combinatorially large, and contains many functionally distinct expressions, as well as

many syntactic rewrites of functionally equivalent expressions.

3. Related Work

Symbolic regression (SR) has traditionally been formulated as a combinatorial optimization problem. While genetic programming (GP) approaches like PySR (Cranmer, 2023) and Operon (Burlacu et al., 2020) remain the gold standard for precision, they treat every task as a new search instance. Amortized methods are instead framed as simulation-based inference (Cranmer et al., 2020): they exploit a massive synthetic training corpus to learn a parameterized approximation of the posterior distribution $p(\tau | \mathcal{D})$ of explanatory expressions τ for given observed data $\mathcal{D}$. However, existing approaches are compromised by the trade-off between expression quality (i.e. simplification) and data diversity and scale.

3.1. Static Corpora and Finite Support

To ensure high-quality training targets, many amortized methods rely on SymPy (Meurer et al., 2017) for simplification. Its computational cost, however, leads many to perform data generation offline, resulting in fixed, pre-computed datasets on the order of 100M expressions (Biggio et al., 2021; Vastl et al., 2024). These expensive and limited training sets ultimately force a trade-off between coverage, diversity, and dimensionality: to maintain a representative density of samples over the prior, training is restricted to limited operator sets and low-dimensional settings ($D \leq 3$). Furthermore, these static datasets are confined to iterating over a fixed subset of the prior, fundamentally restricting the diversity of functional forms the model is exposed to during training compared to on-the-fly expression generation.

3.2. Lack of Normalization and Syntactic Redundancy

To escape the limitations of expensive static datasets, recent works have moved toward on-the-fly procedural generation. However, to maintain training throughput, several methods abandon simplification as a consequence, and train on vast streams of unsimplified expressions (Kamienny et al., 2022; Li et al., 2025). This forces the model to approximate a redundant one-to-many mapping during training where a single underlying function has infinite valid representations (e.g., $x, x+0, 1 \cdot x$) which dilutes the probability mass across syntactically distinct but semantically identical targets. This learned *syntactic redundancy* severely hampers inference efficiency. The generative search dissipates its budget producing syntactic rewrites rather than distinct functional hypotheses, and, since they are not filtered, the constants have to be redundantly re-optimized for each variant.

3.3. The SymPy Bottleneck in Dynamic Generation

A third category of methods attempts to combine on-the-fly generation with CAS-based simplification. NSRwH (Bendinelli et al., 2023) and MDLformer (Yu et al., 2025) apply SymPy within the training loop to simplify generated skeletons. This raises the data quality but introduces a severe computational bottleneck for large scale training. The overhead of object-oriented parsing and tree traversals in SymPy restricts the throughput and maximum complexity of expressions that can be generated in real-time. Consequently, these methods are often limited to lower-dimensional problems.

FLASH-ANSR resolves this dilemma via SIMPLIFY, enabling the high-throughput generation of simplified expressions with high dimensionality and broad operator support, significantly exceeding the scale of prior dynamic methods.

3.4. Methodological Deficiencies in Evaluation

We identify a pervasive lack of rigor in SR concerning data handling and evaluation. Many works report point-estimates without confidence intervals (Bendinelli et al., 2023) or use lenient success thresholds ($R^2 > 0.9$) that mask failures (d’Ascoli et al., 2023). Analyses often ignore inference time budgets, comparing methods with vastly different compute budgets (Valipour et al., 2021). Most alarmingly, we find that with the partial exception of Biggio et al. (2021), virtually no prior work performs rigorous data decontamination. This risks performance overestimation (Carlini et al., 2023) and violates fundamental evaluation principles (Hastie et al., 2009). Our work establishes a strict evaluation protocol: we demand machine-precision recovery ($FVU < 10^{-7}$), analyze the *test-time compute* Pareto frontier with respect to a diverse set of metrics, and perform rigorous and conservative decontamination to prevent test set leakage.

4. Method

We frame SR as a set-to-sequence task in which a transformer encoder-decoder model learns to approximate the posterior distribution $p(\tau | \mathcal{D})$ over expressions (i.e. token sequences in prefix notation) τ given datasets $\mathcal{D}$. Our framework comprises three components: (1) SIMPLIFY, a rule-based engine that reduces algebraic simplification to pattern matching and cancellations; (2) a synchronous, high-throughput data-generation and training pipeline; and (3) a specialized encoder-decoder Transformer.

4.1. SimpliPy: Amortized Symbolic Simplification

To enable high-throughput generation of concise expressions, we introduce SIMPLIFY, a length-reducing rule-based term rewriting system (TRS) $\mathcal{R}$ paired with a multiplicity-based cancellation procedure for additive and multiplicative

subtrees. Training only exposes the model to *normalized* forms, allowing its capacity to be spent on functionally distinct hypotheses rather than on rediscovering algebraic identities. We use the term *normalized form* for the shortest representative of an expression’s algebraic equivalence class under $\mathcal{R}$; among equally-short irreducible forms, we select the one with the fewest constants $n_c(\tau)$. Among the equivalence class $[\tau]_{\mathcal{R}}$ of τ , we select

$$\tau^* = \operatorname{argmin}_{\tau' \in [\tau]_{\mathcal{R}}} (|\tau'|, n_c(\tau')).$$

For example, $x_1 + c$ is the normalized form of the class $\{x_1 + \sin(c), 2x_1 + c_1 + c_2 - x_1, \dots\}$. Standard computer algebra solves this problem from first principles, but this is overkill for the kind of expressions encountered in SR training and therefore unnecessarily slow. Instead, we apply an *amortization* strategy to the simplification process itself and determine normalized forms for (sub)expressions with up to $D = 4$ variables and $L_{\max} = 7$ symbols offline and up-front. Each metavariable $\square_i$ in a rule’s LHS binds to an arbitrary subtree at match time, extending the coverage of these length-bounded patterns to subtrees of arbitrary depth at runtime. This one-time investment allows us to reduce runtime simplification to fast table lookups.

SIMPLIFY operates directly on the tokenized prefix sequence produced by the generator and consumed by the model, allowing us to bypass additional parsing overhead caused by conversions to object-oriented structures.

Phase 1: Amortized Rule Discovery (Offline). We construct $\mathcal{R}$ by stratified enumeration and numeric equivalence testing of patterns and candidate replacements in order of increasing length, similar in spirit to Kruskal’s algorithm (Kruskal, 1956) for minimum spanning forests: each pattern is processed shortest-first and a new rule is added only when $\text{SIMPLIFY}_{\mathcal{R}}$ cannot already reduce that pattern. To permit embarrassingly parallel discovery, $\mathcal{R}$ is held fixed within each length stratum at the cost of a fraction of redundant rules. A formal algorithm is given in Appendix G.

Every accepted rule $\tau \rightarrow \tau'$ satisfies (i) $\text{Vars}(\tau') \subseteq \text{Vars}(\tau)$ (no new variables are introduced) and (ii) $|\tau'| < |\tau|$ (strict length reduction). The discovery loop additionally rejects any rule whose replacement uses a metavariable more often than its LHS, since such rules would not strictly reduce length in general once the metavariable binds to longer subtrees. Together, these conditions make $\mathcal{R}$ a non-duplicating, size-decreasing TRS. Termination follows from the length order $|\cdot|$ being a reduction order. Discovery to $L_{\max} = 7$ has worst-case complexity $O(\alpha^{L_{\text{pattern}} + L_{\text{rep}}})$ in the pattern and replacement lengths and takes ~ 100 h on 32 threads, but is a one-time cost: the pre-computed rule set has been published and is efficiently reusable for every task.

Phase 2: Online Pattern Matching & Cancellation. At runtime, rules are loaded and sorted into operator-specific

buckets keyed by pattern length and root node. Explicit rules (those without variables; *ground* in TRS terminology, $\mathcal{R}_g$) are hash-indexed for $O(1)$ lookup while pattern rules (*non-ground*, $\mathcal{R}_v$) are stored as trees for subtree matching. Given an input expression τ , SIMPLIFY alternates pattern application and term cancellation up to K times (Algorithm 1; full version in Appendix G).

Algorithm 1 SIMPLIFY Simplification (abstract)

```

1: Input: expression  $\tau$ , rule set  $\mathcal{R}$ , max iterations  $K = 5$ ,
   optional max pattern length  $L_{\max}$ 
2:  $\tau_{\text{orig}} \leftarrow \tau$ 
3: for  $i = 1$  to  $K$  do
4:    $\tau \leftarrow \text{CANCELTERMS}(\tau)$ 
5:    $\tau \leftarrow \text{APPLYRULES}(\tau; \mathcal{R}, L_{\max})$ 
6:   if  $\tau$  unchanged then
7:     break
8:   end if
9: end for
10: Sort commutative operands to canonical order; replace
    cancellation-introduced literals with the placeholder for
    free constants  $\diamond$ .
11: return  $\tau$  if  $|\tau| \leq |\tau_{\text{orig}}|$  else  $\tau_{\text{orig}}$ 
    
```

APPLYRULES matches recursively from largest patterns to smallest while CANCELTERMS merges associative-commutative clusters (e.g., $[+, x1, x1] \rightarrow [\text{mult2}, x1]$) and cancels inverse pairs. An optional $L_{\max}$ caps the longest pattern considered, providing control over the speed/quality trade-off.

For an input with n nodes, R rules of maximum pattern length P , and K iterations, the worst-case online complexity is $O(n^2 \cdot (K \cdot R \cdot P + \log n))$ (derivation in Appendix G).

As an example, SIMPLIFY simplifies

$$\left| \frac{x_1}{2} \right|^2 + c_1 \cdot e^{x_2 - x_2} + c_2 \longrightarrow c_1 + \left(\frac{x_1}{2} \right)^2 \quad (1)$$

effectively collapsing the exponential to 1, removing the redundant absolute operator, and substituting $c_1 \leftarrow c_1 + c_2$.

4.2. Architecture

FLASH-ANSR builds on an encoder-decoder Transformer architecture inspired by Biggio et al. (2021). Fully exploiting SIMPLIFY’s data-quality and -scale gains requires a base architecture that scales well to large models and training sets. We therefore depart from Biggio et al. (2021) and propose an improved, scalable architecture (Table 1).

We opt for the pre-RMSNorm Transformer decoder architecture (Vaswani et al., 2017; Zhang & Sennrich, 2019; Xiong et al., 2020) with multi-head FlashAttention (Dao, 2024). Pre-norm stabilizes gradient flow at deeper decoders and

Table 1. Architectural and inference choices in FLASH-ANSR relative to NeSymReS (Biggio et al., 2021).

Component	NeSymReS	FLASH-ANSR
Decoder norm	Post-norm	Pre-norm
Encoder norm	LayerNorm / None	m. RMSSetNorm
Positional enc.	Sinusoidal	RoPE
Input encoding	16-bit IEEE-754	32-bit IEEE-754
Decoding strategy	Beam Search	Softmax Sampling
Inference optimizer	BFGS	Lev.-Marq.

is the de-facto choice for modern Transformer scaling. To encode data sets of variable size, we use a variant of the Set Transformer (Lee et al., 2019) to efficiently encode the input data and use $I = S = 128$ induction and seed points respectively. Inspired by Child et al. (2019); Xiong et al. (2020), and Zhang & Sennrich (2019); Zhang et al. (2022), we introduce the Pre-Norm paradigm to the Set Transformer architecture. We introduce a masked RMSSetNorm based on Zhang & Sennrich (2019); Zhang et al. (2022) to replace the standard LayerNorm (Ba et al., 2016) or SetNorm (Zhang et al., 2022), allowing us to standardize over the same number of axes as SetNorm while only requiring half the number of statistics and re-scaling parameters, and accounting for padding (see Appendix D for derivation). In the decoder we replace sinusoidal positional encodings with RoPE (Su et al., 2023), which injects relative position directly into attention and eliminates the need to recover relative offsets from additive absolute encodings.

As in NeSymReS (Biggio et al., 2021), we preprocess inputs to the Set Transformer by representing each scalar by a multi-hot encoding over a fixed bit representation following the IEEE-754 standard (IEEE, 2008). We use 32 bits instead of just 16 bits to represent each scalar value, leading to an input tensor $\mathbf{Z}^{(0)} \in \mathbb{R}^{N \times M \times 32 \cdot (D+1)}$ with batch size N , broadening the representable range to match physical-domain data (10^{-38} to 10^{38} vs 10^{-4} to 10^4 at 16-bit).

4.3. Data Generation

Training data is generated fully on-the-fly through a four-stage pipeline: (1) skeleton sampling, (2) simplification, (3) decontamination against the test set, and (4) dataset rendering. We illustrate each stage on a single draw and refer to Appendix J for all sampling priors and hyperparameters.

(1) Skeleton sampling. We sample the operator count $n_{\text{ops}} \in \{0, \dots, 17\}$ from a length-exponential prior $P(n_{\text{ops}} = k) \propto \exp(k^{0.7})$, build a prefix skeleton $\bar{\tau}$ (constants masked with placeholders $\diamond$) with the Lamplé & Charton algorithm (Lamplé & Charton, 2020), sample and assign operators to internal nodes (with $\{+, -, *, /\}$ weighted $10\times$ over other operators), and populate the leaves with a multiset of variables and the constant placeholder $\diamond$,

e.g. with $n_{\text{ops}} = 4$:

$$\bar{\tau}^{(1)} = [+, \text{mult2}, \text{sin}, x_1, \text{pow2}, \diamond].$$

(2) Simplification. The candidate is normalized with SIMPLIPY ($L_{\text{max}} = 4$) and rejected if simplification produces non-finite symbols. In our example, SIMPLIPY absorbs `pow2` into the constant placeholder, yielding

$$\bar{\tau}^{(1)*} = [+, \text{mult2}, \text{sin}, x_1, \diamond].$$

(3) Decontamination. We structurally prune all constant nodes from both the candidate and every held-out test skeleton. We then compare the pruned forms symbolically (token equality) and numerically: each pruned form is evaluated on a fixed grid $X_{\text{check}} \sim \mathcal{U}(-10, 10)^{512 \times D}$, rounded element-wise to four decimals, and hashed; the candidate is rejected on hash collision with any held-out output. NaN entries are replaced with 0 before hashing, biasing the filter toward rejection at NaN or near-zero positions. For our example, the pruned form $\bar{\tau}^{(1)*} = [\text{mult2}, \text{sin}, x_1]$ has no test-set match and is accepted for dataset rendering.

(4) Dataset rendering. We sample the number of data points $M \sim \mathcal{U}(1, 1024)$, the per-dimension support range from $\mathcal{N}(0, 10)$, and each constant from $\mathcal{N}(0, 5)$, then compile and evaluate $y = f_{\bar{\tau}}(X; c)$. Datasets containing complex, non-finite, or non-numeric values trigger up to four resampling attempts for $\bar{\tau}$. Persistent failures reject the entire instance. For our example $\bar{\tau}^{(1)*}$, $c_1 = 1.15$, $M = 137$, and the resulting (X, y) is finite-valued and accepted.

4.4. Training

We minimize the cross-entropy loss over the joint distribution of normalized skeletons and datasets $p(\bar{\tau}^*, \mathcal{D})$:

$$\hat{\theta} = \arg \min_{\theta} \mathbb{E}_{\substack{\bar{\tau} \sim p(\bar{\tau}) \\ \mathcal{D} \sim p(\mathcal{D} | \bar{\tau})}} \left[- \sum_{t=1}^L \log p_{\theta}(\bar{\tau}_t^* | \bar{\tau}_{<t}^*, \mathcal{D}) \right] \quad (2)$$

where p_{θ} is the likelihood assigned by the Transformer decoder at step t , conditioned on the previous tokens $\bar{\tau}_{<t}^*$ and the dataset $\mathcal{D}$ encoded by the Set Transformer. Encoder and decoder are trained jointly and end-to-end. We train four models (3M, 20M, 120M, and 1B parameters) using the hyperparameters listed in Appendix B.

4.5. Inference

The posterior $p(\bar{\tau} | \mathcal{D})$ is multi-modal, meaning that multiple distinct functional hypotheses and forms can fit the same data. For diverse candidate generation and better exploration, we replace beam search with softmax sampling and draw K independent candidate skeletons $\bar{\tau}_k \sim p_{\hat{\theta}}(\bar{\tau} | \mathcal{D})$. Integer multiplication and division operator tokens in $\bar{\tau}$ are replaced with multiplications by free constants at a slightly increased

optimization cost to allow for fine-grained numerical optimization of the expression, e.g. $[\dots, \text{mult2}, \cdot, \dots] \rightarrow [\dots, *, \diamond, \cdot, \dots]$. All generated skeletons are then simplified with SimpliPy ($L_{\text{max}} = 4$), deduplicated symbolically, and compiled before their constants are optimized with the Levenberg–Marquardt algorithm (Levenberg, 1944; Marquardt, 1963) under a least-squares objective using SciPy.optimize.curve_fit (Virtanen et al., 2020), yielding optimized expressions $\hat{\tau}$. Empirically, LM achieves ~ 10 pp higher vNRR than BFGS (Broyden, 1970; Fletcher, 1970; Goldfarb, 1970; Shanno, 1970) on FASTSRB at matched compute (Appendix N.2). Optimizations are run in parallel on multiple CPU cores and are restarted 8 times with initial values sampled from $c_0 \sim \mathcal{N}(0, \sigma = 5)$, matching the training prior. Inspired by many other methods (Biggio et al., 2021; Cranmer, 2023), the predicted expressions are then sorted by their fit quality and a parsimony regularization

$$\hat{\tau}^* = \arg \min_{\hat{\tau}} \log_{10} \text{FVU}(\hat{\tau}) + \gamma \cdot |\hat{\tau}| \quad (3)$$

where $\text{FVU}(\hat{\tau})$ is the fraction of variance unexplained (Equation 9) achieved by an expression $\hat{\tau}$, and γ is a small constant penalizing the complexity of a predicted expression. Thus, an expression $\hat{\tau}_b$ is considered better than an expression $\hat{\tau}_a$ if

$$\frac{\text{FVU}(\hat{\tau}_b)}{\text{FVU}(\hat{\tau}_a)} < 10^{-\gamma(|\hat{\tau}_b| - |\hat{\tau}_a|)}. \quad (4)$$

By default, we set $\gamma = 0.05$, meaning that each additional symbol should decrease the FVU by 11% relative to the expression without that symbol.

5. Experiments

We evaluate of FLASH-ANSR against established baselines in amortized SR (NeSymReS, E2E) and direct optimization (PySR) on 115 expressions and data derived from the FASTSRB benchmark (Martinek, 2025)³. Compared to other commonly used benchmarks (e.g. Feynman (Udrescu & Tegmark, 2020), Nguyen (Hoai et al., 2002; Johnson, 2009; Keijzer, 2003; Uy et al., 2011; Petersen et al., 2021)), which restrict the domain of input data to small and arbitrary intervals around zero, FASTSRB adopts the physically motivated data domains of SRSD (Matsubara et al., 2024), ensuring that methods are evaluated on data distributions that reflect real scientific applications.

We evaluate under strict machine-precision recovery metrics and analyze performance as a function of three key scaling axes: test-time compute, data sparsity, and noise robustness.

³We exclude 5 equations (B4, B7, B17, II.24.17, III.14.14) whose prescribed sampling regimes induce `sqrt`-domain or `exp`-overflow issues (see Appendix I for details).

5.1. Time-Normalized Recovery Rate

We take inspiration from Biggio et al. (2021) and evaluate the inference-time-recovery-rate Pareto frontier by sweeping over a characteristic scaling parameter for each method (Table 2), varying the inference time budget from the millisecond regime (< 0.1 s) up to a medium-horizon search (≈ 1000 s) in powers of 2. No artificial restrictions are imposed on hardware acceleration (see Appendix C), and no model is stopped early during inference. For each test expression, we evaluate on 10 distinctly sampled datasets for short-term runs and 5 for medium-horizon runs. Baselines are configured as close to their default settings as possible (see Appendix H for details).

Table 2. Ranges for characteristic inference-time scaling parameters used to analyze performance across orders of magnitude in inference time. Evaluation of our 1B parameter model (Appendix M) is limited to 64k samples to account for additional inference time induced by the larger model size.

METHOD	PARAMETER	RANGES	
		Short ($10\times$)	Medium ($5\times$)
NeSymReS	Beam Width	1 ... 32	128, 512
E2E	Beam Width	1 ... 512	–
PySR	Iterations	1 ... 1024	4096, 16k
Flash-ANSR	Samples	1 ... 16k	64k, 256k

5.2. Data Sparsity

We evaluate the ability of FLASH-ANSR to recover physical laws from sparse data by varying the number of points to fit, $M \in \{2^0, \dots, 2^{11}\}$. To ensure a fair comparison, we fix the inference budget to approximately 10 seconds per expression for all methods. The specific hyperparameters corresponding to this budget are detailed in Table 3.

Table 3. Fixed test time compute settings for the sparse data and noisy data experiments.

METHOD	PARAMETER	VALUE
NeSymReS	Beam Width	4
E2E	Beam Width	256
PySR	Iterations	128
Flash-ANSR v23.0-3M	Samples	4096
Flash-ANSR v23.0-20M	Samples	2048
Flash-ANSR v23.0-120M	Samples	2048
Flash-ANSR v23.0-1B	Samples	512

5.3. Noisy Data

To assess robustness to noise, we introduce additive Gaussian noise to the target variables following a protocol similar to Cava et al. (2021). For the relative noise levels

$\eta \in \{10^{-3}, 10^{-2}, 10^{-1}\}$, we corrupt the data via

$$\tilde{y}_m = y_m + \epsilon_m, \quad \epsilon_m \sim \mathcal{N}(0, \sigma = (\eta \cdot \sigma_y)) \quad (5)$$

where σ_y is the standard deviation of the clean targets.

5.4. Metrics

We assess model performance based on two central metrics capturing the goodness of fit and the parsimony of the predicted expressions.

Relying on exact symbolic match criteria for evaluation is fundamentally flawed for two reasons. Incomplete symbolic canonicalizations of functionally equivalent expressions are not identified as such without expensive manual intervention, leading to misleadingly low recovery rates. Purely symbolic criteria also disregard the vast amount of diverse, functionally distinct expressions that can fit the data equally well as the ground truth. Hence, we focus on numeric recovery as the primary criterion for success.

Numeric Recovery Rate (NRR). Since problems can have widely varying output scales, we base our numeric criterion on the Fraction of Variance Unexplained (FVU) between the ground-truth values y and the values obtained from a predicted expression $\hat{y}$. NRR is the fraction of test *problems* for which the model’s best candidate (Eq. 3 for our models) achieves $\text{FVU}(y, \hat{y}) \leq \epsilon_{32} \approx 1.19 \times 10^{-7}$ (32-bit machine precision). We further distinguish between the recovery rate on the fit data split (fNRR) and an unseen validation data split of equal size (vNRR) for each test instance.

Expression Length Ratio. As a proxy for symbolic quality and interpretability, we evaluate the parsimony of the predicted expressions using the ratio of lengths between predicted and ground truth expressions $|\hat{\tau}|/|\tau|$, where $|\tau|$ denotes the number of nodes in the expression tree of τ .

Total Nestedness. We complement the above with a structural measure, counting the total operator-chaining depth.

Rewrite Fraction. To estimate diversity, we compute the share of generated candidates τ_k per problem that are rewrite-equivalent under SIMPLIFY-normalization.

Bloat. We report $\mathbb{E}_{\tau_k} [|\tau_k|/|\text{SIMPLIFY}(\tau_k)|] - 1$, measuring the inflation of candidates relative to their simplified forms.

We indicate the objective of each metric, e.g. $\text{NRR} \uparrow^{[0,100]}$ and use bootstrapping to estimate the 95% confidence intervals for all statistics (Riezler & Hagmann, 2024).

5.5. Symbolic Simplification Efficiency

We directly compare the effectiveness and efficiency of simplifications with SymPy (Meurer et al., 2017) and SIMPLIFY based on 2^{16} (64k) expressions sampled from our training distribution (Section 4.3). For SymPy, we imple-

ment a compatibility layer that converts our tokenized prefix representation to the corresponding infix string. Then, to work around SymPy’s inherent inability to combine symbols representing free constants, we temporarily substitute randomly sampled values $c_k \sim \mathcal{U}(-10, 10)$ into the expression before parsing it with SymPy, measuring the time of its `simplify` routine, and masking numerical values again with the placeholder $\diamond$. The simplified expression is then converted back to our prefix format with SIMPLIFY’s parser.

6. Results

6.1. Time-Normalized Recovery Rate

We present the results of the test-time normalized performance on the FASTSRB benchmark in Figure 2.

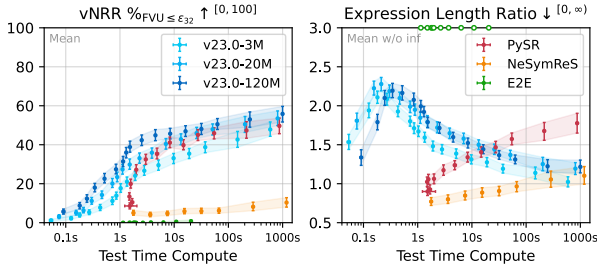


Figure 2. **Left:** Validation Numeric Recovery Rate (vNRR) as a function of inference time (log scale). FLASH-ANSR models (shades of blue) scale monotonically with compute, with the 120M model partially surpassing the PySR baseline (red). Baselines NeSymReS and E2E fail to generalize to the benchmark. **Right:** Expression Length Ratio $|\hat{\tau}|/|\tau|$ versus compute. We observe a *parsimony inversion*: while PySR increases complexity to minimize error over time, FLASH-ANSR converges toward simpler, more canonical expressions as the sampling budget increases. Shaded regions denote 95% confidence intervals.

Baselines. Consistent with the limitations identified in Section 3, prior amortized methods struggle to generalize to the realistic domains of FASTSRB. NeSymReS saturates at a recovery rate of approximately 10%, while E2E fails to achieve meaningful recovery ($< 2.5\%$) across all budgets.

Accuracy Scaling. FLASH-ANSR demonstrates strong power-law scaling with respect to inference compute. While the smallest model (3M) lags behind the genetic programming baseline, the larger variants and the 1B model (Appendix M) effectively bridge the gap. Notably, the 120M parameter model achieves parity with PySR in the medium-compute regime (≈ 10 s) and partially surpasses it at higher budgets, reaching a peak recovery rate of $\sim 58\%$ compared to PySR’s 50.0%. This shows that with sufficient scale and sampling budget, an amortized prior can match or exceed the search efficiency of mature evolutionary algorithms.

The Parsimony Inversion. Crucially, we observe a fundamental divergence in how the methods navigate the

accuracy-complexity trade-off (Figure 2, right). PySR employs a sophisticated model selection strategy: it maintains a Pareto front of the best expressions found at every complexity level and selects the final candidate by maximizing the negative gradient of the log-loss with respect to complexity (finding the “knee” of the curve). Without any parsimony penalty beyond this default strategy, however, PySR exhibits a positive correlation between time and expression complexity: as the evolutionary search progresses, the length ratio increases from 0.94 to 1.85. This suggests that even with parsimony pressure, the GP optimizes residual error by appending corrective terms, resulting in expressions that fit the data via complexity rather than structural discovery.

Conversely, FLASH-ANSR exhibits an *inverse* scaling trend. As the sampling budget increases, the average length ratio of the best-found solution *decreases*, converging towards the ground truth length (e.g., the 120M model improves from 1.40 to 1.27), while simultaneously improving recovery rates. With more compute, the model successfully samples these rarer, concise “needles in the haystack”, rather than constructing complex approximations.

As a realistic application of our method, we present the identification of mathematical test-time scaling laws on the FASTSRB benchmark in Figure 3.

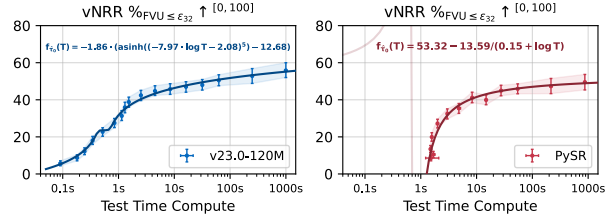


Figure 3. FLASH-ANSR fits to its own and PySR’s scaling curves $\log_{10}(T)$ vs. vNRR from Figure 2 (left) using v23.0-120M, $\gamma = 0.15$, 128k choices ≈ 10 min. Extrapolation suggests an asymptotic vNRR $\propto \log \log T$ scaling for FLASH-ANSR, and an asymptotic upper limit for PySR around 53%.

6.2. Symbolic Simplification Efficiency

Figure 4 characterizes the performance of our SIMPLIFY engine and the SymPy `simplify` routine.

Speedup. Our measurements (Figure 4, left) reveal a significant acceleration with respect to the SymPy baseline which exhibits a median simplification time of approximately 100ms per expression, rendering it prohibitive for real-time applications or large-batch training. In contrast, our pattern-based engine operates in the millisecond regime for $L_{\max} = 4$ (used to train and evaluate FLASH-ANSR).

Quality Preservation. Critically, this speed only comes at a minimal cost of simplification quality. Figure 4 (right) plots the distribution of the simplification ratio $|\tau^*|/|\tau|$. We observe that the distributions for $L_{\max} \geq 5$ even exceed the

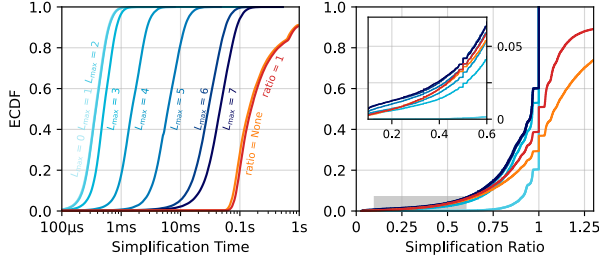


Figure 4. **Left:** Empirical Cumulative Distribution Functions (ECDFs) of simplification wall-clock time. Our SIMPLIFY rewriting engine (shades of blue, varying $L_{\max}$) operates in the low to moderate millisecond regime, orders of magnitude faster than the SymPy baseline (orange, red). **Right:** ECDF of the simplification ratio $|\hat{\tau}|/|\tau|$. The inset highlights the tail of the distribution. Our method with $L_{\max} \geq 5$ achieves simplification ratios comparable to the SymPy baseline while maintaining high throughput.

baseline in terms of conciseness, indicating that our local rewriting rules and cancellations capture the vast majority of reductions found by the more complex heuristics of SymPy. While SIMPLIFY results in strictly shorter or at most equal lengths, SymPy’s simplification paradoxically increases the length of about 38% to 52% of expressions (even with the “ratio” parameter set to 1), and fails to complete within a 1-second timeout for 9% of expressions.

6.3. Data Sparsity

We investigate the impact of data sparsity on model performance by varying the number of support points M from 1 to 2048 (Figure 5). While the amortized baselines (NeSymReS,

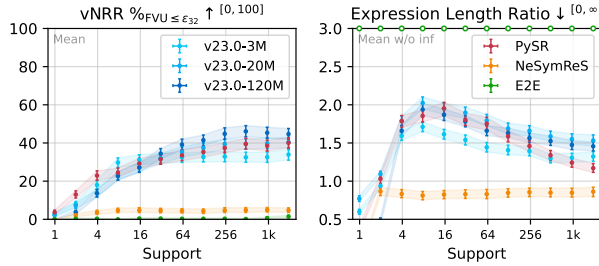


Figure 5. **Left:** Validation Numeric Recovery Rate (vNRR) vs. the number of support points M . FLASH-ANSR (120M) outperforms PySR in the dense data regime ($M > 64$). **Right:** Expression Length Ratio $|\hat{\tau}|/|\tau|$. We observe a distinct “Complexity Peak” at $M \approx 8$, where the model generates expressions significantly longer than the ground truth. This peak coincides with a regime of high uncertainty (low log-probability) and excess constant usage, suggesting the model is *interpolating* the sparse points via complex aliasing rather than identifying the underlying law.

E2E) remain largely ineffective across all regimes, FLASH-ANSR and PySR demonstrate strong scaling behavior.

The structural evolution of the predicted expressions reveals a phase transition analogous to (*Deep*) *Double Descent* (Belkin et al., 2019; Nakkiran et al., 2020). At the critical

threshold of $M \approx 8$ points, we observe a “complexity peak” where the average expression length overshoots the ground truth by up to 2 times. This correlates with a surge in excess constants (using degrees of freedom to minimize the fit error) and a dip in the model’s log-probability (Appendix M.2), suggesting a transition through three distinct regimes:

The Parsimonious Regime ($M < 8$): The data is insufficient to contradict the model’s bias toward simplicity and the scoring function’s preference for parsimonious expressions (Equation 3), resulting in confident, concise approximations with high bias but low variance.

The Interpolation Regime ($M \approx 16$): The data density is sufficient to rule out trivial forms but insufficient to constrain the search to the true symbolic law. To maintain low fit error, the method saturates its complexity budget, inserting excess constants and operators to construct “aliasing” approximations. The drop in log-probability confirms this as a regime of high uncertainty, where probability mass is spread across many complex candidates.

The Identification Regime ($M \rightarrow 2048$): The symbolic cost of explaining the data via complex approximation exceeds the model’s generative capabilities, encouraging a posterior collapse to the ground truth – the only remaining solution that fits the data and is concise enough to be predicted by the model. We observe that distribution shifts in the data can disrupt this delicate balance, preventing successful identification even in the dense data regime.

6.4. Noisy Data

We demonstrate the effects of distribution shifts induced by additive Gaussian noise on the target values in Figure 6.

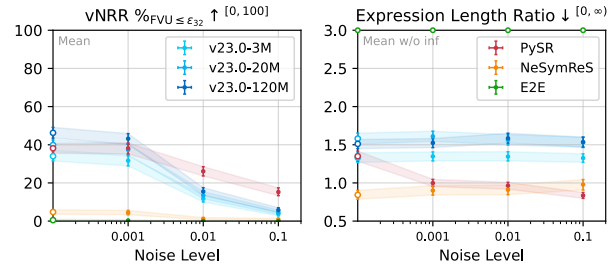


Figure 6. **Left:** Validation Numeric Recovery Rate (vNRR) vs. the noise level η (log scale). While FLASH-ANSR (shades of blue) achieves competitive performance in the noiseless regime (circles), PySR (red) retains higher recovery rates at larger noise levels. **Right:** Expression Length Ratio $|\hat{\tau}|/|\tau|$. On noisy data, PySR (red) and NeSymReS (orange) produce expressions of similar complexity to the ground truth whereas FLASH-ANSR consistently predicts longer expressions without any apparent trend.

In low-noise settings, FLASH-ANSR demonstrates superior performance with the 120M parameter model surpassing the genetic programming baseline PySR (Figure 6, left). At

higher noise levels $\eta \geq 10^{-2}$, we observe a distinct performance crossover. Here, PySR demonstrates better robustness, retaining a higher recovery rate than FLASH-ANSR.

Since FLASH-ANSR was trained exclusively on noiseless data, the injection of noise constitutes a distributional shift. The encoder, having learned that all variance in y carries semantic meaning, misinterprets the noise as high-frequency signals. Consequently, it attempts to “explain” the noise by continuing to generate expressions with rich functional structures, none of which can perfectly fit the intricate variations induced by the noise (Figure 6, right).

6.5. Ablations

We ablate four axes of our deployed v23.0-120M model:

Simplification: We compare performance using SimpliPy, SymPy (Meurer et al., 2017) and no simplification (A-U) during training (10M and 100M expressions) and inference.

Training Prior: We revert the prior for constants from $\mathcal{N}(0, 5)$ to $\mathcal{U}(1, 5)$ used in (Biggio et al., 2021) (C1).

Architecture: We measure architectural contributions by reverting individual components to their prior-work baselines (Biggio et al., 2021): Post-Norm transformers (B1), 16-bit input encoding (B2), and LayerNorm transformers (B3).

Decoding: We compare softmax sampling vs. beam search on the v23.0-120M baseline on equal sample budgets.

Figure 7 highlights the importance of expression simplification (A-U) and full 32-bit input encoding (B2) to achieve concise predictions and high recovery rates.

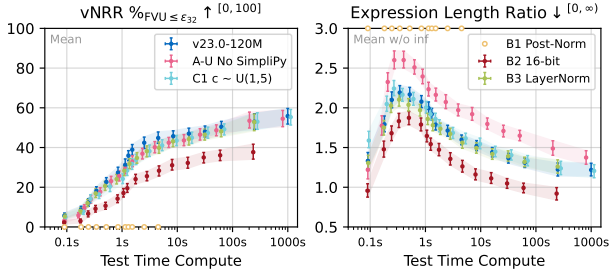


Figure 7. Test-time compute scaling on FASTSRB for ablations A-U, B1, B2, B3, C1 of the 120M model. **Left:** vNRR. Both training-data ablations trail the baseline by a few percentage points up to ~ 10 s of inference and converge at higher budgets. **Right:** Expression Length Ratio. The unsimplified ablation produces persistently longer candidates, settling ~ 40 – 50% above canonical at high compute; the narrow-prior ablation tracks baseline length.

The Post-Norm ablation (B1) training failed due to instability, corroborating our use of the more stable Pre-Norm architecture. Further training prior (C1) and architecture ablations (B1, B3) have more marginal effects with the largest difference in vNRR between 1s and 10s, beyond which all models saturate at similar performance, suggesting that they struggle with the same hard problems.

Figure 8 indicates higher diversity and recovery rates in less time using softmax sampling compared to beam search. This is due to rewrite-mode collapse: at $c=4096$, beam search retains only 33% of its nominal budget as distinct canonical hypotheses (48% for softmax), spending the remainder on syntactic variants of the same dominant canonical form. The deficit concentrates on *short* target equations ($\Delta vNRR \approx +17$ pp); on long ones both decoders explore wide hypothesis sets and the gap shrinks to ~ 2 pp, suggesting beam search’s mode-seeking behavior is most damaging when the posterior is sharply peaked.

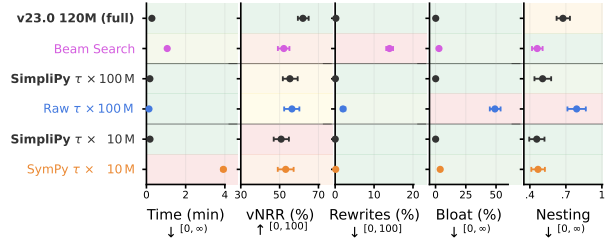


Figure 8. Hypothesis-space fingerprint at fixed training (full, 100M, 10M) and inference volume ($c = 4096$) on FASTSRB. Beam Search yields $70\times$ more equivalent rewrites than softmax sampling and recovers 9.4 pp fewer problems at $4\times$ wall time. Omitting simplification matches SIMPLIPY on vNRR but produces longer expressions (*Bloat*), and $21\times$ more rewrites. SYMPY achieves similar performance as SIMPLIPY at $20\times$ slower inference.

7. Conclusion

We introduced SIMPLIPY, a high-throughput simplification engine that breaks a key bottleneck preventing amortized SR from scaling to scientific complexity, and FLASH-ANSR, which leverages it for large-scale training on diverse, high-quality expressions and efficient inference. On the FASTSRB benchmark, FLASH-ANSR dominates existing amortized baselines and matches state-of-the-art genetic programming (PySR) on the inference-time vs. recovery-rate Pareto frontier, recovering more concise expressions as the inference budget grows. Our ablations (Section 6.5) further reveal that SIMPLIPY’s contribution is a *system-component effect* on hypothesis-space coverage: training-data simplification governs surface-form parsimony and reduces compute on canonical-equivalent rewrites.

Limitations remain, particularly robustness to noisy data, which constitutes an out-of-distribution shift for our models. Future work will incorporate noisy data into training, widen the generated training distributions, and explore alternative encoding and decoding paradigms. By establishing rigorous evaluation standards and demonstrating scalable performance, our work positions amortized SR as a high-speed complement to classical optimization in scientific discovery.

Acknowledgments

This work was funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) within the Priority Programme SPP 2331 (Machine Learning in Chemical Engineering) – Project number 466528284.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Ansel, J., Yang, E., He, H., Gimelshein, N., Jain, A., Voznesensky, M., Bao, B., Bell, P., Berard, D., Burovski, E., Chauhan, G., Chourdia, A., Constable, W., Desmaison, A., DeVito, Z., Ellison, E., Feng, W., Gong, J., Gschwind, M., Hirsh, B., Huang, S., Kalambarkar, K., Kirsch, L., Lazos, M., Lezcano, M., Liang, Y., Liang, J., Lu, Y., Luk, C., Maher, B., Pan, Y., Puhersch, C., Reso, M., Saroufim, M., Siraichi, M. Y., Suk, H., Suo, M., Tillet, P., Wang, E., Wang, X., Wen, W., Zhang, S., Zhao, X., Zhou, K., Zou, R., Mathews, A., Chanan, G., Wu, P., and Chintala, S. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pp. 929–947. ACM, 2024. doi: 10.1145/3620665.3640366.
- Ba, J. L., Kiros, J. R., and Hinton, G. E. Layer normalization, 2016. URL <https://arxiv.org/abs/1607.06450>.
- Belkin, M., Hsu, D., Ma, S., and Mandal, S. Reconciling modern machine-learning practice and the classical bias-variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854, 2019. doi: 10.1073/pnas.1903070116.
- Bendinelli, T., Biggio, L., and Kamienny, P.-A. Controllable neural symbolic regression, 2023. URL <https://arxiv.org/abs/2304.10336>.
- Biggio, L., Bendinelli, T., Neitz, A., Lucchi, A., and Parascandolo, G. Neural symbolic regression that scales. In *International Conference on Machine Learning*, pp. 936–945. Pmlr, 2021.
- Broyden, C. G. The convergence of a class of double-rank minimization algorithms. *Journal of the Institute of Mathematics and Its Applications*, 6(1):76–90, 1970. doi: 10.1093/imamat/6.1.76.
- Burlacu, B., Kronberger, G., and Kommenda, M. Operon C++: An efficient genetic programming framework for symbolic regression. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion, GECCO ’20*, pp. 1562–1570, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450371278. doi: 10.1145/3377929.3398099. URL <https://doi.org/10.1145/3377929.3398099>.
- Carlini, N., Ippolito, D., Jagielski, M., Lee, K., Tramèr, F., and Zhang, C. Quantifying memorization across neural language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=P282diP8Cq>.
- Cava, W. G. L., Orzechowski, P., Burlacu, B., de França, F. O., Virgolin, M., Jin, Y., Kommenda, M., and Moore, J. H. Contemporary symbolic regression methods and their relative performance. In Vanschoren, J. and Yeung, S.-K. (eds.), *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021*, 2021.
- Child, R., Gray, S., Radford, A., and Sutskever, I. Generating long sequences with sparse transformers. *ArXiv*, abs/1904.10509, 2019. URL <https://api.semanticscholar.org/CorpusID:129945531>.
- Cranmer, K., Brehmer, J., and Louppe, G. The frontier of simulation-based inference. *Proceedings of the National Academy of Sciences*, 117(48):30055–30062, May 2020. ISSN 1091-6490. doi: 10.1073/pnas.1912789117. URL <http://dx.doi.org/10.1073/pnas.1912789117>.
- Cranmer, M. Interpretable machine learning for science with PySR and SymbolicRegression.jl, 2023. URL <https://arxiv.org/abs/2305.01582>.
- Dao, T. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations, ICLR 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=mZn2Xyh9Ec>.
- d’Ascoli, S., Becker, S., Mathis, A., Schwaller, P., and Kilbertus, N. ODEFormer: Symbolic regression of dynamical systems with transformers. *arXiv preprint arXiv:2310.05573*, 2023.
- Fletcher, R. A new approach to variable metric algorithms. *The Computer Journal*, 13(3):317–322, 1970. doi: 10.1093/comjnl/13.3.317.

- Goldfarb, D. A family of variable-metric methods derived by variational means. *Mathematics of Computation*, 24(109):23–26, 1970. doi: 10.1090/S0025-5718-1970-0258249-6.
- Gunasekar, S., Zhang, Y., Aneja, J., Mendes, C. C. T., Giorno, A. D., Gopi, S., Javaheripi, M., Kauffmann, P., de Rosa, G., Saarikivi, O., Salim, A., Shah, S., Behl, H. S., Wang, X., Bubeck, S., Eldan, R., Kalai, A. T., Lee, Y. T., and Li, Y. Textbooks are all you need, 2023. URL <https://arxiv.org/abs/2306.11644>.
- Hägele, A., Bakouch, E., Kosson, A., Allal, L. B., von Werra, L., and Jaggi, M. Scaling laws and compute-optimal training beyond fixed training durations. *ArXiv*, abs/2405.18392, 2024. URL <https://api.semanticscholar.org/CorpusID:270068102>.
- Hastie, T., Tibshirani, R., and Friedman, J. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, NY, 2 edition, 2009.
- Hoai, N. X., McKay, R. I., Essam, D., and Chau, R. Solving the symbolic regression problem with tree-adjunct grammar guided genetic programming: The comparative results. In *Proceedings of the 2002 Congress on Evolutionary Computation. CEC’02 (Cat. No. 02TH8600)*, volume 2, pp. 1326–1331. IEEE, 2002.
- Hu, S., Tu, Y., Han, X., He, C., Cui, G., Long, X., Zheng, Z., Fang, Y., Huang, Y., Zhao, W., Zhang, X., Thai, Z. L., Zhang, K., Wang, C., Yao, Y., Zhao, C., Zhou, J., Cai, J., Zhai, Z., Ding, N., Jia, C., Zeng, G., Li, D., Liu, Z., and Sun, M. MiniCPM: Unveiling the potential of small language models with scalable training strategies, 2024. URL <https://arxiv.org/abs/2404.06395>.
- IEEE. IEEE standard for floating-point arithmetic. *IEEE Std 754-2008*, pp. 1–70, 2008. doi: 10.1109/IEEESTD.2008.4610935.
- Johnson, C. G. Genetic programming crossover: Does it cross over? In *Genetic Programming: 12th European Conference, EuroGP 2009 Tübingen, Germany, April 15-17, 2009 Proceedings 12*, pp. 97–108. Springer, 2009.
- Kamienny, P.-A., d’Ascoli, S., Lample, G., and Charton, F. End-to-end symbolic regression with transformers. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems 35*, 2022.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models, 2020. URL <https://arxiv.org/abs/2001.08361>.
- Keijzer, M. Improving symbolic regression with interval arithmetic and linear scaling. In *European Conference on Genetic Programming*, pp. 70–82. Springer, 2003.
- Koza, J. R. Genetic programming as a means for programming computers by natural selection. *Statistics and Computing*, 4:87–112, 1994. URL <https://api.semanticscholar.org/CorpusID:8750149>.
- Kruskal, J. B. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1):48–50, 1956. ISSN 00029939, 10886826. URL <http://www.jstor.org/stable/2033241>.
- Lample, G. and Charton, F. Deep learning for symbolic mathematics. In *8th International Conference on Learning Representations, ICLR 2020*. OpenReview.net, 2020. URL <https://openreview.net/forum?id=S1eZYeHFDS>.
- Lee, J., Lee, Y., Kim, J., Kosiorek, A., Choi, S., and Teh, Y. W. Set transformer: A framework for attention-based permutation-invariant neural networks. In Chaudhuri, K. and Salakhutdinov, R. (eds.), *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 3744–3753. PMLR, 2019. URL <https://proceedings.mlr.press/v97/lee19d.html>.
- Lee, K., Ippolito, D., Nystrom, A., Zhang, C., Eck, D., Callison-Burch, C., and Carlini, N. Deduplicating training data makes language models better, 2022. URL <https://arxiv.org/abs/2107.06499>.
- Levenberg, K. A method for the solution of certain non – linear problems in least squares. *Quarterly of Applied Mathematics*, 2:164–168, 1944. URL <https://api.semanticscholar.org/CorpusID:124308544>.
- Li, Y., Liu, J., Wu, M., Yu, L., Li, W., Ning, X., Li, W., Hao, M., Deng, Y., and Wei, S. MMSR: Symbolic regression is a multi-modal information fusion task. *Information Fusion*, 114:102681, February 2025. ISSN 1566-2535. doi: 10.1016/j.inffus.2024.102681. URL <http://dx.doi.org/10.1016/j.inffus.2024.102681>.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *7th International Conference on Learning Representations, ICLR 2019*. OpenReview.net, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- Marquardt, D. W. An algorithm for least-squares estimation of nonlinear parameters. *Journal of the Society for Industrial and Applied Mathematics*, 11(2):431–441, 1963.

- doi: 10.1137/0111030. URL <https://doi.org/10.1137/0111030>.
- Martinek, V. Fast symbolic regression benchmarking, 2025. URL <https://arxiv.org/abs/2508.14481>.
- Matsubara, Y., Chiba, N., Igarashi, R., and Ushiku, Y. Rethinking symbolic regression datasets and benchmarks for scientific discovery, 2024. URL <https://arxiv.org/abs/2206.10540>.
- Meurer, A., Smith, C. P., Paprocki, M., Čertík, O., Kirpichev, S. B., Rocklin, M., Kumar, A., Ivanov, S., Moore, J. K., Singh, S., Rathnayake, T., Vig, S., Granger, B. E., Muller, R. P., Bonazzi, F., Gupta, H., Vats, S., Johansson, F., Pedregosa, F., Curry, M. J., Terrel, A. R., Roučka, v., Saboo, A., Fernando, I., Kulal, S., Cimrman, R., and Scopatz, A. SymPy: symbolic computing in Python. *PeerJ Computer Science*, 3:e103, January 2017. ISSN 2376-5992. doi: 10.7717/peerj-cs.103. URL <https://doi.org/10.7717/peerj-cs.103>.
- Nakkiran, P., Kaplun, G., Bansal, Y., Yang, T., Barak, B., and Sutskever, I. Deep double descent: Where bigger models and more data hurt. In *8th International Conference on Learning Representations, ICLR 2020*. OpenReview.net, 2020. URL <https://openreview.net/forum?id=Blg5sA4twr>.
- Petersen, B. K., Landajuela, M., Mundhenk, T. N., Santiago, C. P., Kim, S. K., and Kim, J. T. Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients. In *Proc. of the International Conference on Learning Representations*, 2021.
- Riezler, S. and Hagmann, M. *Validity, Reliability, and Significance: Empirical Methods for NLP and Data Science*. Synthesis Lectures on Human Language Technologies. Springer Cham, 2024. ISBN 978-3-031-57065-0. doi: 10.1007/978-3-031-57065-0. URL <https://doi.org/10.1007/978-3-031-57065-0>.
- Schmidt, M. D. and Lipson, H. Distilling free-form natural laws from experimental data. *Science*, 324:81 – 85, 2009. URL <https://api.semanticscholar.org/CorpusID:7366016>.
- Shanno, D. F. Conditioning of quasi-Newton methods for function minimization. *Mathematics of Computation*, 24(111):647–656, 1970. doi: 10.1090/S0025-5718-1970-0274029-X.
- Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. RoFormer: Enhanced transformer with rotary position embedding, 2023. URL <https://arxiv.org/abs/2104.09864>.
- Udrescu, S.-M. and Tegmark, M. AI Feynman: A physics-inspired method for symbolic regression. *Science Advances*, 6(16):eaay2631, 2020. doi: 10.1126/sciadv.aay2631.
- Uy, N. Q., Hoai, N. X., O’Neill, M., McKay, R. I., and Galván-López, E. Semantically-based crossover in genetic programming: application to real-valued symbolic regression. *Genetic Programming and Evolvable Machines*, 12:91–119, 2011.
- Valipour, M., You, B., Panju, M., and Ghodsi, A. SymbolicGPT: A generative transformer model for symbolic regression. *arXiv preprint arXiv:2106.14131*, 2021.
- Vastl, M., Kulhánek, J., Kubalík, J., Derner, E., and Babuška, R. SymFormer: End-to-end symbolic regression using transformer-based architecture. *IEEE Access*, 12:37840–37849, 2024. doi: 10.1109/ACCESS.2024.3374649.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In Guyon, I., von Luxburg, U., Bengio, S., Wallach, H. M., Fergus, R., Vishwanathan, S. V. N., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 30*, pp. 5998–6008, 2017.
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., Carey, C. J., Polat, İ., Feng, Y., Moore, E. W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E. A., Harris, C. R., Archibald, A. M., Ribeiro, A. H., Pedregosa, F., van Mulbregt, P., and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., and Liu, T. On layer normalization in the transformer architecture. In III, H. D. and Singh, A. (eds.), *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 10524–10533. PMLR, 2020. URL <https://proceedings.mlr.press/v119/xiong20b.html>.
- Yu, Z., Ding, J., Li, Y., and Jin, D. Symbolic regression via MDLformer-guided search: from minimizing prediction error to minimizing description length, 2025. URL <https://arxiv.org/abs/2411.03753>.
- Zhang, B. and Sennrich, R. Root mean square layer normalization. *ArXiv*, abs/1910.07467, 2019. URL <https://api.semanticscholar.org/CorpusID:113405151>.

Zhang, K. and Shasha, D. Simple fast algorithms for the editing distance between trees and related problems. *SIAM journal on computing*, 18(6):1245–1262, 1989.

Zhang, L., Tozzo, V., Higgins, J., and Ranganath, R. Set norm and equivariant skip connections: Putting the deep in deep sets. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., and Sabato, S. (eds.), *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 26559–26574. PMLR, 17–23 Jul 2022. URL <https://proceedings.mlr.press/v162/zhang22ac.html>.

A. Examples

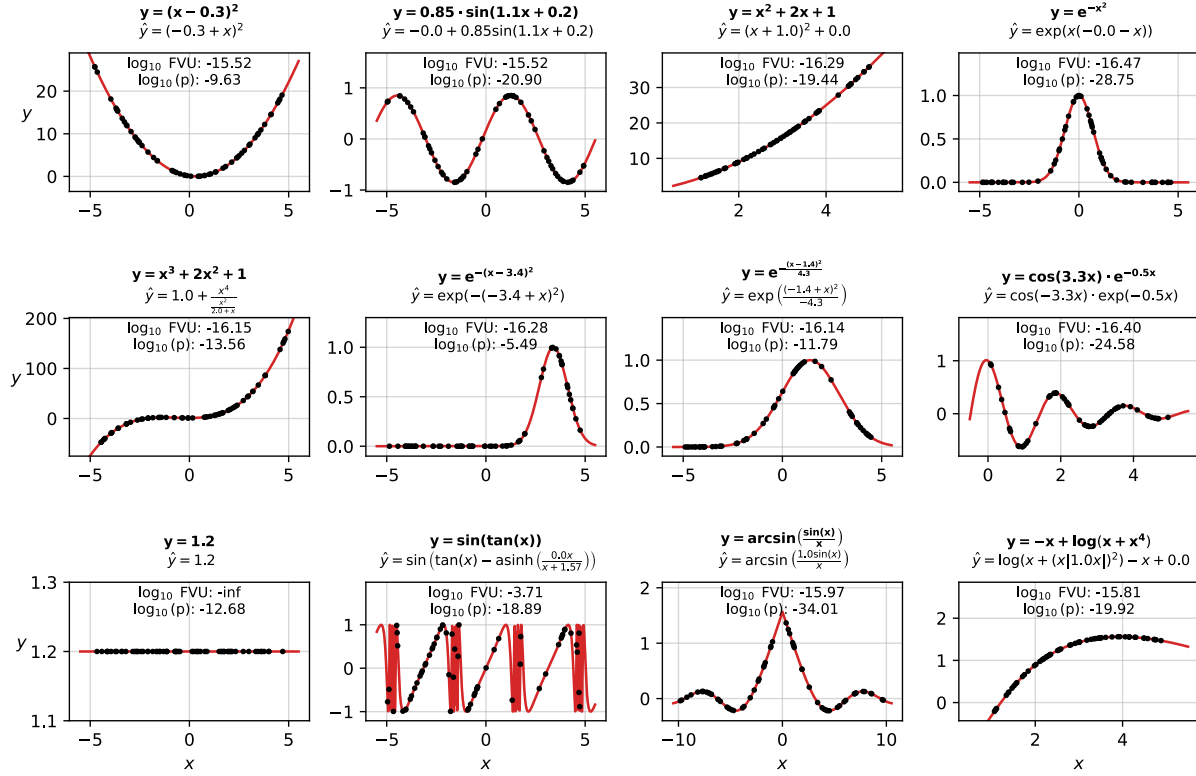


Figure 9. FLASH-ANSR fits (v23.0-120M, $\gamma = 0.05$, 32k samples ≈ 100 s) (red curves) on uniformly sampled data points (black dots) from several 1D expressions ($M = 64$, $\eta = 0$). Many ground truth expressions (bold) are recovered exactly, while others are recovered in alternative forms (e.g. $\hat{y} = (x + 1)^2$ for $y = x^2 + 2x + 1$).

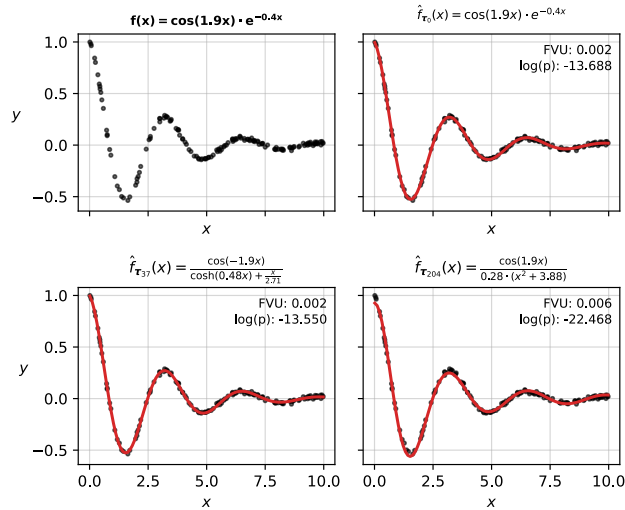


Figure 10. Diverse FLASH-ANSR solutions (v23.0-120M, $\gamma = 0.05$, 32k samples ≈ 100 s) (red curves) on uniformly sampled data points (black dots) depicting the trajectory of a damped harmonic oscillator ($M = 150$, $\eta \approx 0.04$). Multiple distinct expressions fit the data equally well given the noisy data. The ground truth expression (top left, bold) is exactly recovered with high probability (top right). Other solutions (bottom row) achieve similar fit quality while differing structurally.

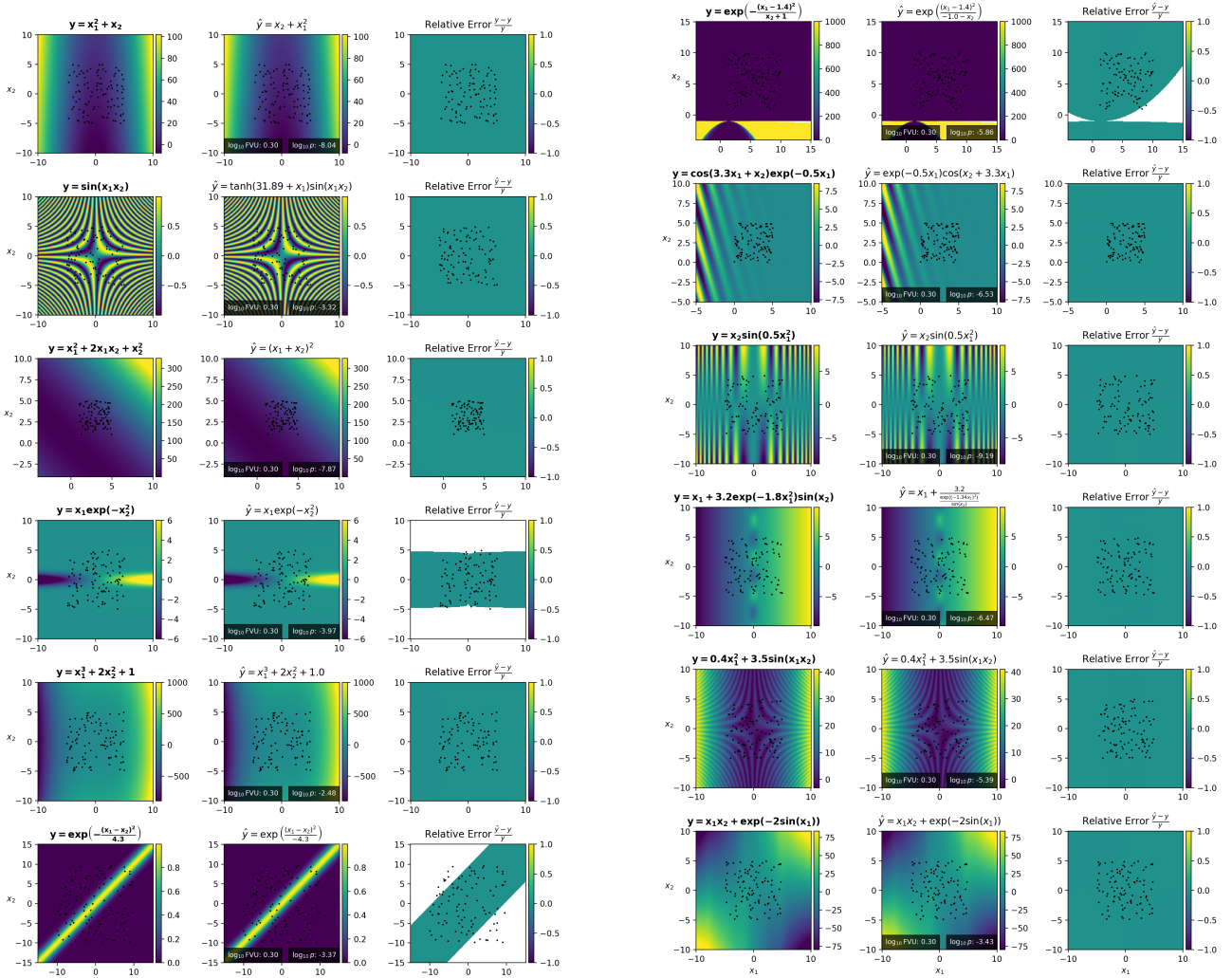


Figure 11. FLASH-ANSR fits (v23.0-120M, $\gamma = 0.05$, 32k samples ≈ 100 s) (middle columns) on uniformly sampled data points (black dots) from several 2D expressions from Biggio et al. (2021) ($M = 100$, $\eta = 0$). Many ground truth expressions (left columns, bold) are recovered exactly, while a few others have additional terms that do not affect the relative fit error (right columns).

B. Hyperparameters

We provide a detailed specification of the hyperparameters and training configurations in Table 4.

Training. All models are trained end-to-end using the AdamW optimizer (Loshchilov & Hutter, 2019) with $\beta_1 = 0.9$. We utilize a Warmup-Stable-Decay learning rate schedule (Hu et al., 2024; Hägele et al., 2024) with a linear warmup from 0 to $\eta_{\max}$, a constant phase, and a linear decay to 0. The 3M, 20M, and 120M models use a 10%/70%/20% split while the 1B model uses an extended 18.75%/56.25%/25% split. We apply gradient clipping with a maximum global norm of 2.0.

Architecture & Implementation. Our models are implemented in PyTorch 2.9. To maximize training throughput, we use Automatic Mixed Precision (AMP) with `torch.compile` (Ansel et al., 2024). The AMP dtype is bf16 on Ada / Ampere / Hopper GPUs (RTX 4090, A100, H200) and fp16 with loss scaling on the Turing RTX 2080Ti used for the 3M model. We apply dropout with $p = 0.1$ to attention weights and to the output of feed-forward networks. In the decoder, we utilize Rotary Positional Embeddings (RoPE) (Su et al., 2023) applied to the queries and keys of the self-attention mechanism. The Set Transformer encoder uses $I = 128$ inducing points and $S = 128$ seed vectors across all model sizes.

Simplification. We employ our SIMPLIFY symbolic simplification engine (version dev_7-3) with a maximum pattern length of $L_{\max} = 4$ during training and inference.

Table 4. Hyperparameters for the Flash-ANSR model series.

HYPERPARAMETER	v23.0-3M	v23.0-20M	v23.0-120M	v23.0-1B
<i>Encoder</i>				
Dimension d_{enc}	192	384	640	1280
Heads	3	6	10	20
ISAB Layers	1	2	5	10
SAB Layers	1	2	5	10
<i>Decoder</i>				
Dimension d_{dec}	192	384	640	1280
Heads	3	6	10	20
Layers	3	6	10	20
Feed Forward Dim	576	1152	1920	3840
<i>Training</i>				
Optimizer Steps	1M	2M	3M	4M
Learning Rate η_{max}	10^{-4}	10^{-4}	10^{-4}	$3 \cdot 10^{-5}$
Weight Decay	0.01	0.01	0.01	0.025
β_2	0.95	0.95	0.95	0.99995
Logical CPUs	4	32	8	8
GPU	RTX 2080Ti	RTX 4090	A100	H200
Time Taken	112h	56h	340h	657h
<i>Parameters</i>				
Total	3.2M	23M	122M	955M
Encoder	1.6M	10.4M	64.4M	494M
Decoder	1.6M	12.4M	57.5M	459M

C. Test-Time Compute System

All runtime-sensitive experiments for Section 6.1 (Time-Normalized Recovery Rate) were conducted on a single representative high-end consumer workstation hardware setup (Table 5) to ensure comparability.

Table 5. System specifications for Time-Normalized Recovery Rate experiment series.

COMPONENT	SPECIFICATION
CPU	AMD Ryzen 9 9950X (16C, 32T)
GPU	NVIDIA RTX 4090 (24 GB)
RAM	64 GB DDR5 6000MT/s
OS	Ubuntu 24.04.3 LTS
Python	3.13.7
PyTorch	2.9.1 + CUDA 13.0

D. Masked RMSNorm for Sets

Let $Z \in \mathbb{R}^{N \times M \times d}$ be a tensor representing a batch of N sets, each containing up to M points with dimension d padded with arbitrary values where necessary. Let $\Xi \in \{0, 1\}^{N \times M}$ be a binary mask where $\Xi_{n,m} = 1$ indicates a valid point and 0 indicates padding. We introduce a learnable scaling parameter $\beta \in \mathbb{R}^d$. For each set $n \in \{1, \dots, N\}$, we compute the masked root mean square statistic ν_n over all valid points and features:

$$\nu_n = \sqrt{\frac{\sum_{m=1}^M \sum_{j=1}^d (Z_{n,m,j})^2 \cdot \Xi_{n,m}}{\max(1, d \cdot \sum_{m=1}^M \Xi_{n,m})}} + \epsilon \quad (6)$$

where ϵ is a small constant for numerical stability. The normalized output $\hat{Z}$ is then obtained by scaling the input element-wise:

$$\hat{Z}_{n,m,j} = \frac{Z_{n,m,j}}{\nu_n} \cdot \beta_j \quad (7)$$

E. Model Architecture

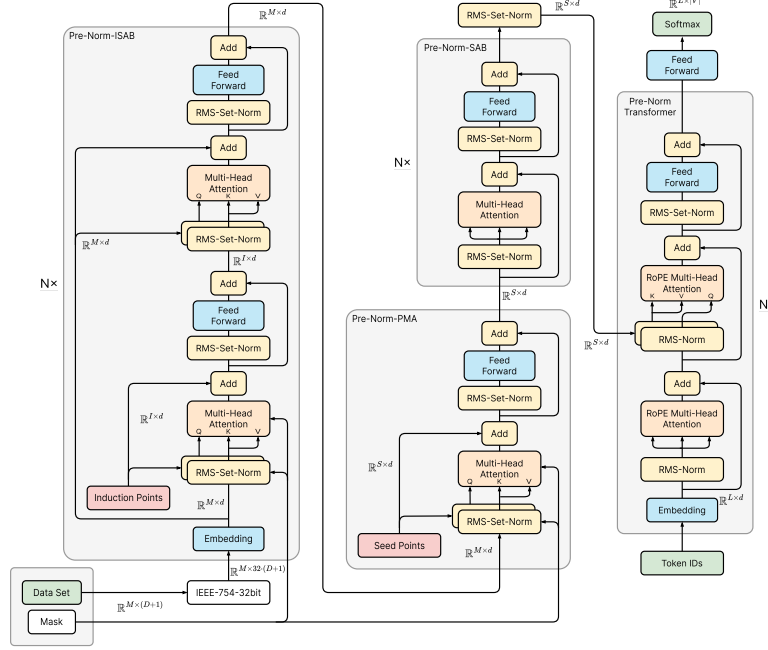


Figure 12. Visual depiction of the Flash-ANSR model architecture. The Set Transformer encoder ingests a variable-sized set of input-output pairs and produces a fixed-size latent representation via Induced Set Attention Blocks (ISAB) and Set Attention Blocks (SAB). The Transformer decoder autoregressively generates a symbolic expression token-by-token, attending to the encoded dataset at each step.

F. Operators

The set of operators supported by FLASH-ANSR and SIMPLIFY is summarized in Table 6. During training, we sample $+$, $-$, $*$, and $/$ with weight 10 and all others with weight 1.

Table 6. Flash-ANSR operator vocabulary.

TOKEN	DESCRIPTION	TOKEN	DESCRIPTION	TOKEN	DESCRIPTION
$+$	Addition	pow1_2	$\sqrt{x}$	asinh	$\operatorname{arcsinh}(x)$
$-$	Subtraction	pow1_3	$\sqrt[3]{x}$	acosh	$\operatorname{arccosh}(x)$
$*$	Multiplication	pow1_4	$\sqrt[4]{x}$	atanh	$\operatorname{arctanh}(x)$
$/$	Division	pow1_5	$\sqrt[5]{x}$	exp	e^x
abs	$ x $	sin	$\sin(x)$	log	$\log(x)$
inv	$\frac{1}{x}$	cos	$\cos(x)$	mult2	$2 \cdot x$
neg	$-x$	tan	$\tan(x)$	mult3	$3 \cdot x$
pow	Exponentiation	asin	$\arcsin(x)$	mult4	$4 \cdot x$
pow2	x^2	acos	$\arccos(x)$	mult5	$5 \cdot x$
pow3	x^3	atan	$\arctan(x)$	div2	$\frac{x}{2}$
pow4	x^4	sinh	$\sinh(x)$	div3	$\frac{x}{3}$
pow5	x^5	cosh	$\cosh(x)$	div4	$\frac{x}{4}$
		tanh	$\tanh(x)$	div5	$\frac{x}{5}$

G. Algorithms

We provide formal versions of the algorithms used by SIMPLIFY, in two passes matching Section 4.1’s phase split. We open with the offline rule-discovery procedure (Algorithm 2) and the numerical equivalence check (Algorithm 3) it invokes, together with a discussion of $\mathcal{R}$ ’s non-minimality (Appendix G.1); we then turn to the online subroutines CANCELTERMS (Algorithm 4) and APPLYRULES (Algorithm 5) that compose the abstract simplification loop of Algorithm 1 and their joint complexity analysis.

Algorithm 2 SimpliPy Rule Discovery

```

1: Input: Operators  $\mathcal{O}$  (Appendix F), Variables  $\mathcal{V}$ , Literals  $\mathcal{C}$  (Section 4.1), Limits  $L_{\max} = 7, L_{\text{rep}} = 3$ , Data  $X \sim \mathcal{N}(0, 5)^{1024 \times D}$ , number of challenges  $K = 16$  and retries  $R = 16$ 
2: Initialize rule set  $\mathcal{R} \leftarrow \emptyset$  and pre-compute  $\mathcal{E}_L$ , the set of expressions of length  $L$  using  $\mathcal{O}, \mathcal{V}, \mathcal{C}$  for  $L = 1$  to  $L_{\max}$ 
3: for length  $L = 1$  to  $L_{\max}$  do
4:   Initialize temporary rule cache  $\mathcal{R}_{\text{new}} \leftarrow \emptyset$ 
5:   for each expression  $\tau \in \mathcal{E}_L$  do
6:      $\tau^* \leftarrow \text{SIMPLIFY}_{\mathcal{R}}(\tau)$ ; found  $\leftarrow \text{False}$ ;  $\tau'_{\text{best}} \leftarrow \text{None}$ 
7:     if  $|\tau^*| < |\tau|$  then
8:       continue {Kruskal-style skip:  $\tau$  already reduces under  $\mathcal{R}$ ; no new rule needed}
9:     end if
10:    for length  $j = 1$  to  $\min(|\tau^*| - 1, L_{\text{rep}})$  do
11:      for each  $\tau' \in \mathcal{E}_j$  where  $\text{Vars}(\tau') \subseteq \text{Vars}(\tau)$  do
12:        if Equivalent( $\tau, \tau'; X, K, R$ ) then
13:          if  $\tau'_{\text{best}} = \text{None}$  or  $n_c(\tau') < n_c(\tau'_{\text{best}})$  then
14:             $\tau'_{\text{best}} \leftarrow \tau'$ 
15:          end if
16:          found  $\leftarrow \text{True}$ 
17:        end if
18:      end for
19:      if found and no metavariable in  $\tau'_{\text{best}}$  occurs more often than in  $\tau$  then
20:         $\mathcal{R}_{\text{new}} \leftarrow \mathcal{R}_{\text{new}} \cup \{\tau \rightarrow \tau'_{\text{best}}\}$ ; break
21:      end if
22:    end for
23:  end for
24:   $\mathcal{R} \leftarrow \mathcal{R} \cup \text{DEDUPLICATE}(\mathcal{R}_{\text{new}})$  {under variable-canonicalization}
25:  Rehash explicit rules into  $\mathcal{R}_g$ ; rebucket pattern rules into  $\mathcal{R}_v$  by (ROOT, length)
26: end for
27: return  $\mathcal{R}$ 
    
```

G.1. Design Choices in Rule Discovery

The discovery procedure of Algorithm 2 fixes $\mathcal{R}$ for the duration of each length- L pass: new rules accumulate in a side cache $\mathcal{R}_{\text{new}}$ and are merged into $\mathcal{R}$ only after the pass completes. This makes the inner loop embarrassingly parallel (every $\tau \in \mathcal{E}_L$ is tested against the same snapshot of $\mathcal{R}$, with no inter-worker coordination) and is what allows the 32-thread discovery run to complete in ~ 100 h. In exchange, distinct expressions in $\mathcal{E}_L$ cannot see rules just admitted within the same pass, so a fraction of the rules in the published $\mathcal{R}$ are subsumed by other rules from the same length stratum and could be removed without changing $\text{SIMPLIFY}_{\mathcal{R}}$ on any input.

A minimal example illustrates the situation. At $L = 3$, $\tau_W = x_0 \cdot 0$ and $\tau_E = 1 \cdot 0$ are both irreducible under $\mathcal{R}$ at the start of the pass: CANCELTERMS counts ADD/MULT multiplicities but does not encode zero as a multiplicative annihilator. The inner search finds $\tau' = 0$ equivalent to each, so the pass admits the pattern rule $W : \square_1 \cdot 0 \rightarrow 0$ from τ_W (which carries a variable) and the explicit rule $E : 1 \cdot 0 \rightarrow 0$ from τ_E (which does not). After commit, W matches $t \cdot 0$ for any subterm t , including $t = 1$; removing E leaves $\text{SIMPLIFY}_{\mathcal{R}}$ unchanged.

The redundancy is strictly intra-length. A length- L_2 metapattern with $L_2 > L_1$ requires a matched subterm of length $\geq L_2$ and so cannot subsume any length- L_1 rule, and when a length- L_2 rule is admitted, every shorter rule is already in $\mathcal{R}$ and

Algorithm 3 Expression Equivalence Check

```

1: Input: pattern  $\tau$ , candidate replacement  $\tau'$ , data  $X$ , challenges  $K = 16$ , retries  $R = 16$ , tolerance  $\epsilon$  (caller-specified)
2: Let  $C$  be constants of  $\tau$ ;  $C'$  be constants of  $\tau'$ 
3: Convention: allclose( $a, b, \epsilon$ ) treats NaN entries as matches (equal_nan=True); the conservative bias from this
   convention is what makes the caller's decontamination filter err on the side of rejection.
4: for  $k = 1$  to  $K$  do
5:   Sample  $r \sim \mathcal{N}(0, \sigma = 5)^{|C|}$ 
6:   for  $s \in \{-1, 0, 1\}^{|C|}$  do
7:      $y_\tau \leftarrow f_\tau(X; |r| \odot s)$ 
8:     if  $|C'| = 0$  then
9:        $\text{match} \leftarrow \text{allclose}(f_{\tau'}(X), y_\tau, \epsilon)$ 
10:    else
11:       $\text{match} \leftarrow \text{False}$ 
12:      for  $j = 1$  to  $R$  do
13:         $p_0 \sim \mathcal{N}(0, \sigma = 5)^{|C'|}$ 
14:        if  $\exists \hat{c}$  s.t. allclose( $f_{\hat{\tau}}(X; \hat{c}), y_\tau, \epsilon$ ) with Levenberg–Marquardt initialized at  $p_0$  then
15:           $\text{match} \leftarrow \text{True}$ ; break
16:        end if
17:      end for
18:    end if
19:    if  $\neg \text{match}$  then
20:      return False
21:    end if
22:  end for
23: end for
24: return True

```

has been applied in $\text{SIMPLIFY}_{\mathcal{R}}(\tau)$ before any new rule is considered. Same-length subsumption forces the substitution between the two metapatterns to be length-preserving, which collapses each metavariable to a single token as illustrated.

A removal-and-recheck pruning pass would eliminate the residual redundancy at $O(R \cdot T_{\text{SIMPLIFY}})$ additional time. The deployment cost of the extra rules is small: the online complexity bound $O(K \cdot n^2 \cdot R \cdot P + n^2 \log n)$ absorbs them linearly in R , and bucketing of $\mathcal{R}_v$ by root operator and pattern length further dampens the constant in practice.

Algorithm 4 CANCELTERMS: multiplicity-based cancellation in additive and multiplicative subtrees.

```

1: Input: expression  $\tau$  in prefix notation
2:  $T \leftarrow \text{PARSETREE}(\tau)$ 
3: Classes: ADD = ( $\{+, -\}$ , neutral 0, hyperoperator MULT); MULT = ( $\{*, /\}$ , neutral 1, hyperoperator POW). Inverse operators ( $-$ ,  $/$ ) flip the polarity of their right operand.
4: Pass 1. bottom-up multiplicity annotation:
5: for each subtree  $s$  in  $T$ , visited bottom-up do
6:   for each class  $C \in \{\text{ADD}, \text{MULT}\}$  do
7:      $(n_s^{C,+}, n_s^{C,-}) \leftarrow$  multiplicities of  $s$  as a positive / negative operand within the enclosing class- $C$  chain (counted across nested operators of the same class; inverse-operator polarity flips applied at right operands)
8:   end for
9: end for
10: Pass 2. select cancellation target + rewrite:
11: Pick the first  $(s^*, C^*)$  in bottom-up order with  $n_{s^*}^{C^*,+} + n_{s^*}^{C^*,-} > 1$ , skipping composite operands containing the constant placeholder  $\diamond$ .
12: Let  $m \leftarrow n_{s^*}^{C^*,+} - n_{s^*}^{C^*,-}$ .
13: if  $m = 0$  then
14:   replace every occurrence of  $s^*$  in the  $C^*$ -chain with the class-neutral element
15: else if  $|m| = 1$  then
16:   keep the first occurrence (with sign  $\text{sgn}(m)$ ); replace the rest with the neutral element
17: else
18:   replace the first occurrence with hyperoperator $_{|m|}(s^*)$ , prefixed by the class-inverse operator iff  $\text{sgn}(m) < 0$ ; replace the rest with the neutral element {if  $|m|$  exceeds the available hyperoperator token range, fall back to a binary op with a numeric coefficient/exponent}
19: end if
20: return FLATTEN( $T$ )
    
```

G.2. Complexity Analysis

We derive the worst-case complexity of online simplification (Algorithm 1 including Algorithms 4, 5 as subroutines) for an input expression with n tokens, a rule set $\mathcal{R}$ of R rules with maximum pattern length P , and the iteration cap K .

Subroutine 1: Cancellation (Algorithm 4 above). Cancellation runs in two stack-based passes over the prefix tree. The bottom-up multiplicity pass visits each token once, maintaining per-class multiplicity tables on a stack of subtree annotations. Merging at each binary node touches only that node’s operands, so each subtree contributes a constant amount of work along its root-to-leaf path. The top-down rewrite pass selects subtrees with $n^+ + n^- > 1$ and rewrites them, touching a bounded number of operands per subtree. Thus, the combined cost is $O(n)$ per call.

Subroutine 2: Rule application (Algorithm 5 below). Per node, REWRITE performs (i) a constant-time hash lookup in explicit rules $\mathcal{R}_g$, then (ii) iterates over the pattern rule buckets $\mathcal{R}_v[(\text{ROOT}(t), |p|)]$ for $|p|$ from P down to 1, with each match attempt costing $O(P)$ for syntactic matching and substitution. The worst-case bucket size is R (loose bound), so per-node matching is $O(R \cdot P)$. Without rule matches, the recursion visits each of the m nodes in the input subtree a constant number of times (initial Pass-1 attempt and post-children retry), giving $O(m \cdot R \cdot P)$.

When rules match, each application strictly reduces token length by the size-decreasing property of Section 4.1, and the substituted replacement is recursively rewritten by a fresh REWRITE call. Let the chain of recursive calls produced by a single top-level invocation have subtree sizes $m = m_0 > m_1 > \dots > m_q \geq 1$. The total cost of the chain is

$$\sum_{k=0}^q m_k \cdot R \cdot P \leq R \cdot P \cdot \sum_{k=0}^{q-1} (n - k) = O(n^2 \cdot R \cdot P).$$

This bound is achieved when each rewrite reduces length by exactly one and the entire shrinking expression is re-traversed by the next REWRITE call.

Algorithm 5 APPLYRULES: pattern matching against the discovered rule set.

```

1: Input: expression  $\tau$ , explicit rules  $\mathcal{R}_g$  (hash-indexed by prefix sequence), pattern rules  $\mathcal{R}_v$  (bucketed by key
   (ROOT( $t$ ),  $|p|$ ); within each bucket, rules are iterated in decreasing  $|p|$ )
2:  $T \leftarrow \text{PARSETREE}(\tau)$ ;  $T \leftarrow \text{REWRITE}(T)$ 
3: return FLATTEN( $T$ )
4:
5: subroutine REWRITE( $t$ ):
6: if  $t$  is a leaf then
7:   return  $t$ 
8: end if
9: if every operand of  $t$  is the constant placeholder  $\diamond$  then
10:  return  $\diamond$  {constant folding}
11: end if
12: Pass 1. try to rewrite the current node:
13: if  $\exists r : (t \rightarrow r) \in \mathcal{R}_g$  then
14:  return REWRITE( $r$ )
15: end if
16: for rules  $(p \rightarrow r) \in \mathcal{R}_v[(\text{ROOT}(t), |p|)]$  in decreasing  $|p|$  do
17:   if a substitution  $\sigma$  exists with  $p[\sigma] = t$  (syntactic matching on the prefix representation) then
18:    return REWRITE( $r[\sigma]$ )
19:   end if
20: end for
21: Pass 2. recurse into children, then retry:
22: for each child  $c$  of  $t$  do
23:    $c \leftarrow \text{REWRITE}(c)$ 
24: end for
25: let  $t'$  be the resulting node; retry Pass 1 on  $t'$  (explicit lookup, then pattern bucket)
26: return  $t'$ 
    
```

Iteration cap. The outer loop of Algorithm 1 runs at most K times, with early termination on convergence. Across K iterations, cancellation and rule application accumulate to $O(K \cdot n^2 \cdot R \cdot P)$ (rule application dominates).

Operand sorting. The final canonicalization pass runs once after the iteration loop. It sorts commutative-operator chains lexicographically by a recursive structural key. With up to n comparable operands in a single chain, each pairwise key comparison costs $O(n)$ for full subtree traversal in the absence of cached hashes, and a sort makes $O(n \log n)$ comparisons. Total: $O(n^2 \log n)$.

In total, the worst-case complexity of online simplification is

$$T_{\text{online}}(n, R, P, K) = O(K \cdot n^2 \cdot R \cdot P + n^2 \log n). \quad (8)$$

The first term captures rule-driven simplification across K iterations, the second the one-time canonical-ordering pass.

Practical Notes. The $n^2 \log n$ term is tight for fully commutative expressions (e.g. a single $+$ chain over n leaves). The $K \cdot n^2 \cdot R \cdot P$ term is tight when rule applications cascade and the entire shrinking expression is re-traversed by the recursive REWRITE call. Bucketing of $\mathcal{R}_v$ by $(\text{ROOT}(t), |p|)$ does not improve the worst-case bound (a single bucket can in principle contain all R rules) but reduces the practical constant: rules distribute across $O(P \cdot |\mathcal{O}|)$ buckets, so observed bucket sizes are far below R . Most expressions converge in 1–3 iterations, well below the $K = 5$ cap.

H. Baselines

Baseline methods are configured at or as close to their defaults as possible while matching the problem setting.

For PySR, we allow the same operator set as FLASH-ANSR, excluding explicit integer multiplication and division operators.

For NeSymReS, we evaluate the “100M” model trained on 100M expressions with the default 4 optimizer restarts.

For E2E, we employ the released `model1.pt` with default refinement parameters (10 trees, 100 bags, 200 points per bag). To attempt evaluation at higher inference budgets (beam width ≥ 512), we apply a dynamic reduction of the maximum generation length to satisfy internal batching constraints. We could not get E2E to function beyond a beam width of 512.

For both, we apply *minimal* compatibility patches to the official implementation to ensure basic functionality in the evaluation environment. These patches are documented and made publicly available alongside our codebase.

I. Excluded FastSRB Equations

We exclude 5 of the 120 FASTSRB equations (B4, B7, B17, II.24.17, III.14.14) whose prescribed sampling regimes induce `sqrt`-domain or `exp`-overflow issues that the benchmark’s $100\times$ resampling protocol fails to resolve. Concretely, the prescribed input domains for these equations either include negative values inside a `sqrt` or exponents large enough to overflow 32-bit floats with high probability, and resampling produces the same family of issues across attempts. We treat this as a property of the SRSD/FASTSRB sampling specification rather than of the SR methods evaluated against it: every method we evaluate fails uniformly on these problems for the same data-domain reason, regardless of inference budget.

J. Data Generation Hyperparameters

We list the priors and hyperparameters of the on-the-fly data-generation pipeline introduced in Section 4.3.

J.1. Skeleton Sampling

- Operator count: $n_{\text{ops}} \sim p_{\text{ops}}(n_{\text{ops}}) \propto \exp[n_{\text{ops}}^\alpha / \lambda]$ with $\alpha = 0.7$, $\lambda = 1$, support $n_{\text{ops}} \in \{0, 1, \dots, 17\}$.
- Maximum skeleton length: 35 symbols.
- Operator weights: $\{+, -, *, /\}$ at relative weight 10, with all other operators (Appendix F) at weight 1.
- Leaf cardinality: number of unique variables $n_{\text{var}^*} \sim \mathcal{U}(1, \min(n_{\text{leaves}}, D))$, where n_{leaves} is the number of leaves in the prefix tree and D is the considered maximum dimensionality.
- Variable subset: drawn uniformly without replacement from $\mathcal{V} \cup \{\diamond\}$, then duplicated and shuffled to fill n_{leaves} positions.

J.2. Simplification

SIMPLIFY is applied to the sampled skeletons with $L_{\text{max}} = 4$. Those containing non-finite tokens after simplification (`float("inf")`, `float("-inf")`, `float("nan")`) are rejected.

J.3. Decontamination

Constants are stripped from both candidate and held-out skeletons before comparison. Symbolic equality of pruned forms triggers immediate rejection. Otherwise, both skeletons are evaluated on a fixed grid $X_{\text{check}} \sim \mathcal{U}(-10, 10)^{512 \times D}$ sampled once per worker at startup, the resulting images are rounded element-wise to four decimal places, and a candidate is rejected on hash collision with any held-out image. NaN entries are replaced with 0 before hashing, biasing the filter toward rejection at NaN or near-zero positions.

J.4. Dataset Rendering

- Number of data points: $M \sim \mathcal{U}(1, 1024)$.
- Per-dimension support range: $a_j, b_j \sim \mathcal{N}(0, \sigma = 10)$ truncated to $[-30, 30]$.
- Per-dimension data points: $X_j \sim \mathcal{U}(a_j, b_j)$.
- Constants: $c_k \sim \mathcal{N}(0, \sigma = 5)$ i.i.d. per constant.
- Validity guard: instances yielding complex or non-finite $y = f_{\bar{\tau}}(X; c)$ are resampled from the same skeleton up to four times before the entire instance is rejected.

Statistics of the resulting training distribution are in Appendix K.

K. Data Statistics

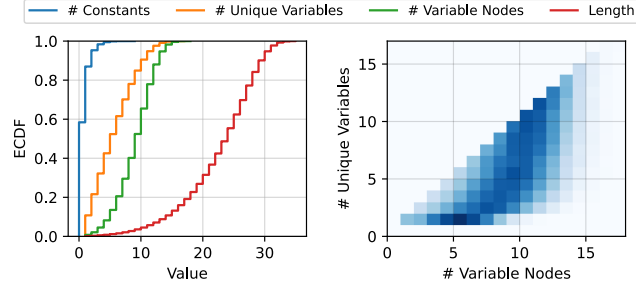


Figure 13. Training data statistics. Left: Empirical Cumulative Distribution Functions (ECDFs) of the number of constants, unique variables, variable nodes and the expression length in the training data. Half of the simplified training expressions contain at least 5 unique variables and 24 symbols. Right: 2D histogram showing the number of unique variables and the number of variable nodes. The training expressions cover both low and high-dimensional regimes with complexity arising both due to the coupling of like and unlike variables.

L. Training Dynamics and Scaling

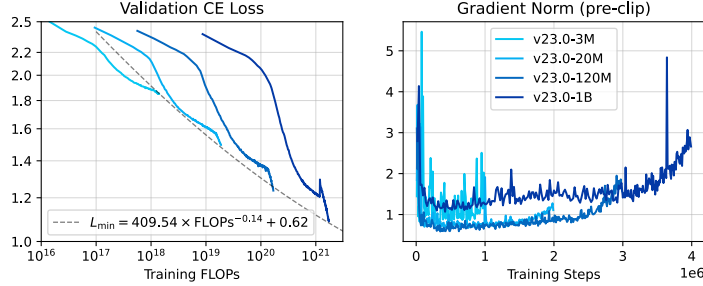


Figure 14. Training dynamics and scaling behavior. **Left:** Validation Cross-Entropy loss (log-scale) as a function of estimated training FLOPs (log-scale). We use the approximation $\text{FLOPs} = 6ND$ following Kaplan et al. (2020), and count FLOPs separately for the encoder with its dataset input and the decoder with its tokenized expression input before adding both for the total FLOPs. The performance envelope generally follows a power-law scaling relationship ($L \propto \text{FLOPs}^{-0.14}$). **Right:** Evolution of gradient norms (pre-clipping). Optimization is stable across the 3M–120M regime. The 1B model exhibits one gradient spike late in training from which it fully recovers over the subsequent training steps. Further investigation is required to determine if this is a systemic issue at large model sizes.

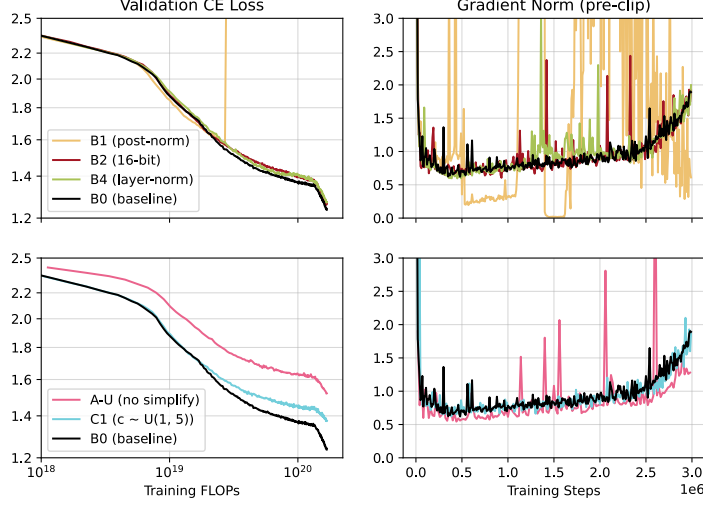


Figure 15. Training dynamics of the system-effect ablations against $B0$ (black, shown in both rows), a configuration-matched retrain of v23.0-120M. **Top:** Architecture axis (B-track). $B1$ (Post-Norm decoder), $B2$ (16-bit input encoding), $B3$ (LayerNorm encoder). **Bottom:** Training-data axis (A-track / C-track). $A-U$ (unsimplified training expressions), $C1$ (narrow $\mathcal{U}(1, 5)$ constants prior). **Left:** Validation Cross-Entropy loss vs. estimated training FLOPs. **Right:** Gradient norms (pre-clipping) vs. training steps. $B1$ (Post-Norm) shows a substantial loss spike around 2×10^{19} FLOPs and sustained gradient instability (alternating between vanishing-gradient regions and large spikes from $\sim 500k$ steps onward). $B2$ (16-bit input encoding) and $B3$ (LayerNorm encoder) track the baseline more closely with only occasional gradient spikes. The training-data ablations exhibit stable optimization but converge to higher loss floors than the baseline ($A-U$ most pronounced). We list detailed per-ablation metrics in Appendix N.

M. Additional Metrics

We use the following definition for the Fraction of Variance Unexplained (FVU):

$$\text{FVU}(y, \hat{y}) = \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} \quad (9)$$

with $\bar{y} = \frac{1}{N} \sum_{i=1}^N y_i$.

Beyond the primary results, we report the following diagnostics with 95% confidence intervals derived from 1000-fold bootstrapping.

Log₁₀ FVU (non-perfect fits). For cases that are *not* numerically perfect ($\text{FVU} > \epsilon_{32}$), we summarize $\log_{10}(\text{FVU})$. Exact fits are excluded before aggregation.

Numeric Recovery Rate (NRR). For a benchmark with N_{test} problems, NRR is the fraction of problems where the predicted expression achieves a numerically perfect fit on the data points $y^{(i)} \in \mathbb{R}^{M \times D}$ of each problem i :

$$\text{NRR} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \mathbb{I} \left(\text{FVU}(y^{(i)}, \hat{y}^{(i)}) \leq \epsilon_{32} \right) \quad (10)$$

with $\epsilon_{32} \approx 1.19 \times 10^{-7}$ being the machine epsilon for 32-bit floating point numbers. We report fNRR and vNRR for the fit split and an unseen validation split of equal size.

Symbolic Recovery Rate (SRR). Exact structural match between simplified predicted and ground-truth prefix skeletons:

$$\text{SRR} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \mathbb{I} \left(\text{SIMPLIFY}(\hat{\tau}^{(i)}) = \text{SIMPLIFY}(\bar{\tau}^{(i)}) \right). \quad (11)$$

Expression F1. Token-level F1 between the predicted prefix skeleton and the simplified ground-truth skeleton.

ZSS Tree Edit Distance. Zhang–Shasha tree edit distance (Zhang & Shasha, 1989) between predicted and ground-truth skeleton trees.

Variable Identification. Recall on the *set of unique variables* present in the simplified predicted vs. ground-truth expressions.

Log Probability. Mean log-probability of the generated sequence under the model.

Excess Constants. Difference in the number of constants between prediction and ground truth. Positive values indicate an overuse of constants to fit the data.

Expression Length Ratio. Length of the predicted prefix skeleton divided by the length of the simplified ground-truth skeleton.

Total Nestedness. Sum of consecutive nestings contributed by unary operators over the predicted expression as a proxy for naturalness and interpretability. For example, $y = \sin(\log(\operatorname{arccosh}(x)))$ has a total nestedness of 2, and $y = \sin(\log(x_1 + \diamond))$ has a total nestedness of 1.

Rewrite Fraction. For each test problem the model returns K candidate predictions $\{\hat{\tau}_k\}_{k=1}^K$. Two candidates are *rewrite-equivalent* if their SIMPLIFY-normalized prefix skeletons are identical. The Rewrite Fraction is the share of candidates that collide under this normalization, averaged across problems:

$$\text{Rewrites} = \mathbb{E}_{\text{problem}} \left[1 - \frac{|\{\text{SIMPLIFY}(\hat{\tau}_k) : k = 1, \dots, K\}|}{K} \right]. \quad (12)$$

Higher values indicate rewrite-mode collapse: the budget is spent on algebraic restatements of the same canonical form rather than on functionally distinct hypotheses.

Bloat. For a candidate $\hat{\tau}$ with SIMPLIFY-normalized form $\text{SIMPLIFY}(\hat{\tau})$, Bloat is the average relative inflation of raw outputs over their simplified equivalents, minus one (so canonical outputs score 0):

$$\text{Bloat} = \mathbb{E}_{\hat{\tau}} \left[\frac{|\hat{\tau}|}{|\text{SIMPLIFY}(\hat{\tau})|} \right] - 1. \quad (13)$$

A value of 50% corresponds to raw predictions $1.5\times$ as long as their canonical forms on average. Unlike Expression Length Ratio, Bloat requires no ground-truth reference: it is computed entirely from the model’s outputs and the simplification engine.

Relative Fit Time. Fit-time multiplier relative to the hyperparameter configuration for the test-time compute experiment.

Fit Success. Fraction of test cases where the fitting procedure completed without errors.

M.1. Detailed Test Time Scaling Results

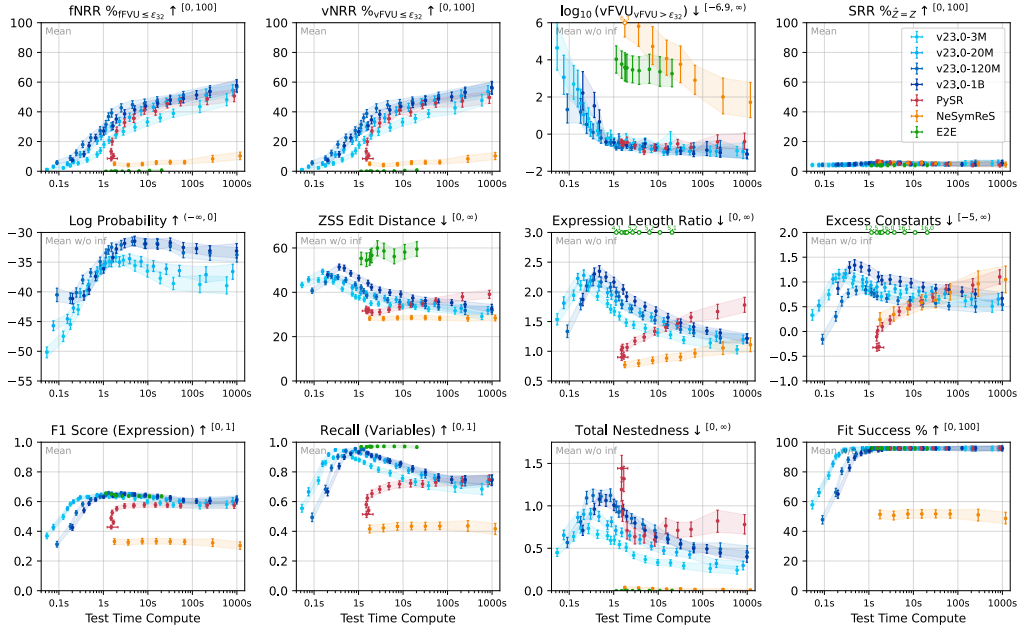


Figure 16. Detailed comparison of time-normalized numerical and symbolic metrics. With increasing time budget, FLASH-ANSR consistently generates more concise (lower expression length ratio), specific (fewer excess constants), and natural (lower total nestedness) expressions than PySR.

M.2. Detailed Data Sparsity Results

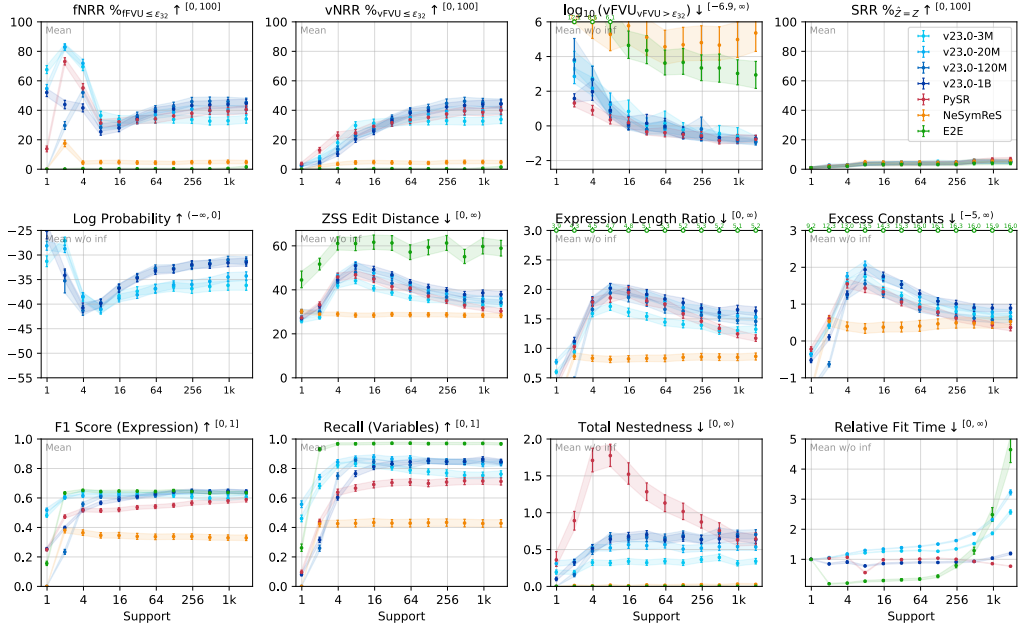


Figure 17. Detailed results for varying number of data points M on the FASTSRB benchmark. In the Interpolation Regime ($N \approx 16$), FLASH-ANSR maintains a lower mean total nestedness than PySR.

M.3. Detailed Noise Level Scaling Results

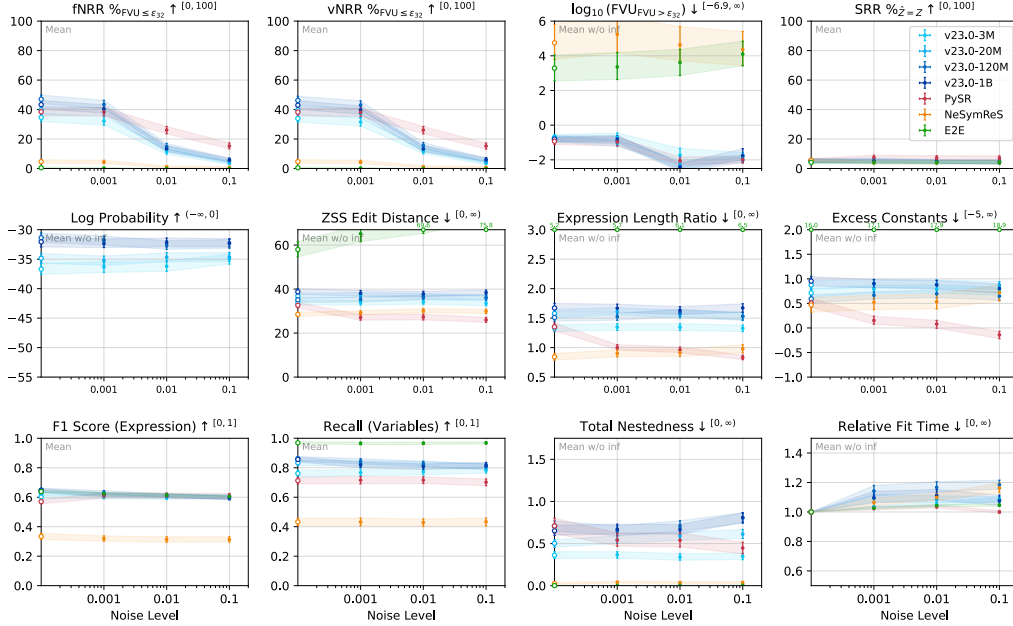


Figure 18. Detailed results for varying noise levels η on the FASTSRB benchmark. Across all noise levels, FLASH-ANSR’s expressions maintain similar quality. With higher noise levels, PySR still achieves shorter and more specific expressions.

N. Ablations

We ablate four axes of the deployed v23.0-120M model:

- **Training data:** A-U (no SIMPLIFY on training expressions) and C1 (narrow $\mathcal{U}(1, 5)$ constants prior).
- **Architecture:** B1 (Post-Norm decoder), B2 (16-bit input encoding), B3 (LayerNorm encoder).
- **Refiner:** BFGS in place of Levenberg–Marquardt (Appendix N.2).
- **Decoding:** softmax sampling vs. beam search (Section 6.5).

Each variant perturbs exactly one axis and shares architecture, training data, optimizer, and 3M-step compute budget with the deployed v23.0-120M. The training-data, prior, and architecture variants additionally share their training batch with B0, a configuration-matched retrain of v23.0-120M used as the matched control for the training-dynamics plot (Figure 15). Test-time accuracy comparisons (Figure 19) use the deployed v23.0-120M.

Constant optimization in our inference pipeline (Section 4.5) uses the Levenberg–Marquardt (LM) algorithm via `scipy.optimize.curve_fit`. We re-run the FastSRB evaluation on the deployed 120M baseline with LM replaced by BFGS (`scipy.optimize.minimize, method=BFGS`), holding all other pipeline components (generation, decontamination, ranking) identical.

Figure 20 shows the full multi-metric comparison plotted against per-problem wall time. At every compute budget LM achieves higher recovery (fNRR/vNRR), higher fit success, lower validation FVU, lower expression-length ratios, and higher F1 score than BFGS; the gap is most visible at moderate compute (1–10 s) and stabilizes around $\Delta vNRR \approx +11$ pp at large budgets. Matching BFGS’s vNRR at $c = 8192$ ($\approx 39\%$) places LM between $c = 128$ and $c = 512$, an order-of-magnitude reduction in compute at matched recovery.

N.1. Detailed Ablation Metrics

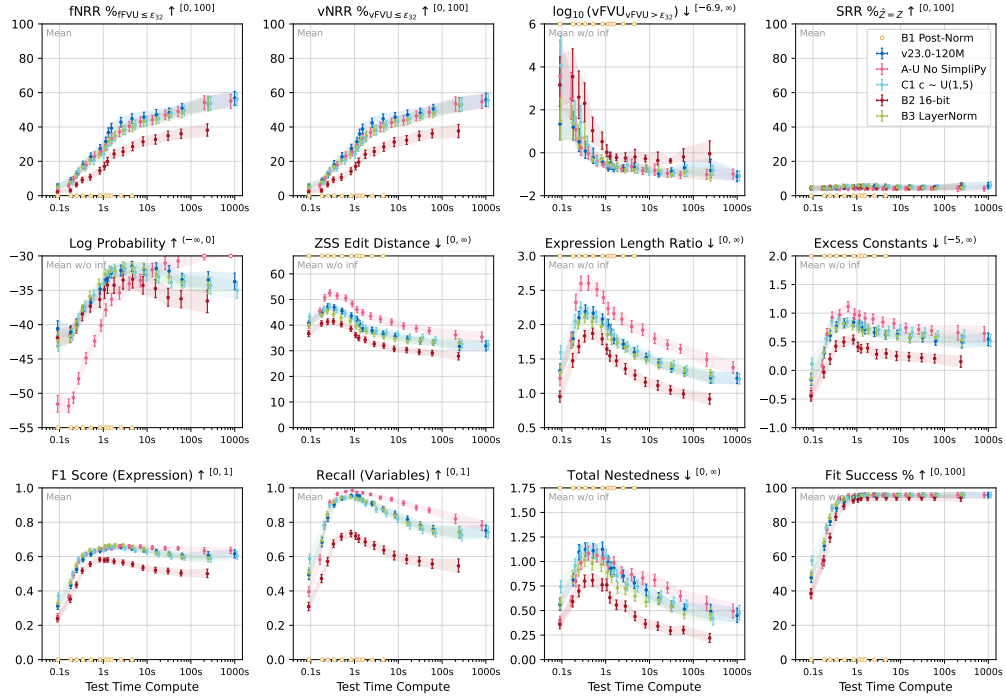


Figure 19. Time-normalized numerical and symbolic metrics for the five system-effect ablations (A-U, B1, B2, B3, C1) against the deployed v23.0-120M baseline. A-U and B2 move recovery and parsimony metrics most. B1 fails completely due to unstable training.

N.2. Refiner Choice: BFGS vs. Levenberg–Marquardt

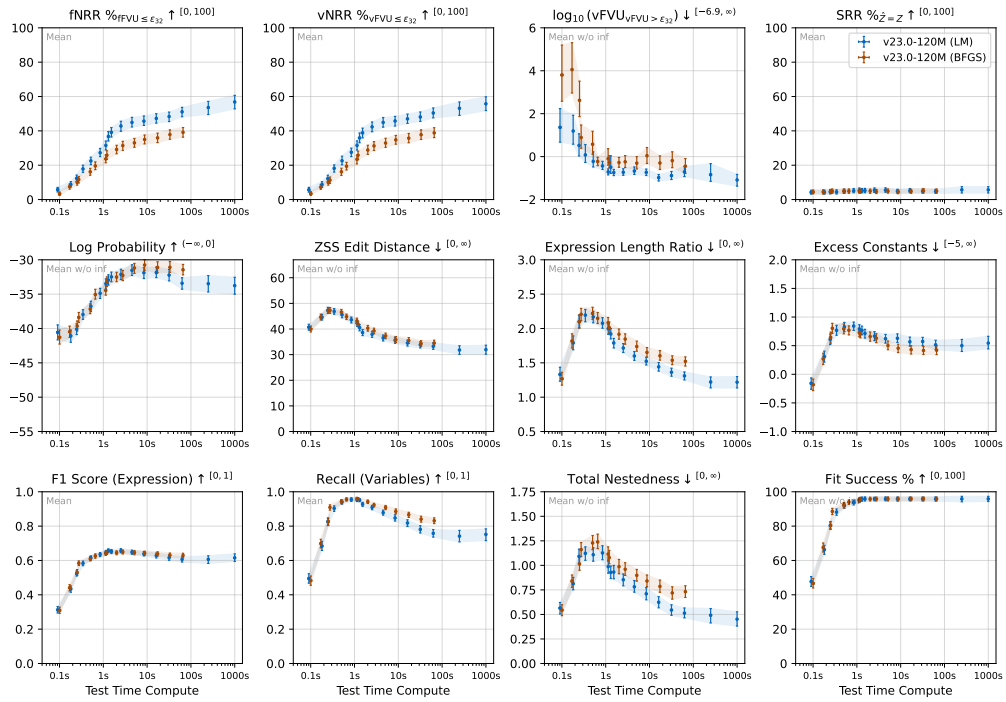


Figure 20. Time-normalized numerical and symbolic metrics for the BFGS vs. Levenberg–Marquardt (LM) refiner ablation. LM dominates BFGS at every budget ($\Delta vNRR \approx +11$ pp at large compute).

N.3. Constants Prior on the Gaussian Family

On a Gaussian $f(x; \mu, \sigma) = \exp(-\frac{1}{2}(x - \mu)^2/\sigma^2)$ we sample $\mu \sim \mathcal{U}(-15, 15)$ and $\sigma \sim \text{LogUniform}(10^{-3}, 10^2)$, and fit 2,048 instances per (model, budget) pair with FLASH-ANSR. At 512 candidates both priors leave failure regions in the extreme- $|\mu|$ low- σ corners ($|\mu| \approx 12, \sigma \approx 0.02$), where the Gaussian peak occupies a needle-thin window in the data: the broader $\mathcal{N}(0, 5)$ prior fails roughly symmetrically in μ , whereas the narrow $\mathcal{U}(1, 5)$ prior of Biggio et al. (2021) fails more on the negative- μ side, coinciding with its skewed prior. At 8,192 candidates both priors converge across most of the (μ, σ) plane.

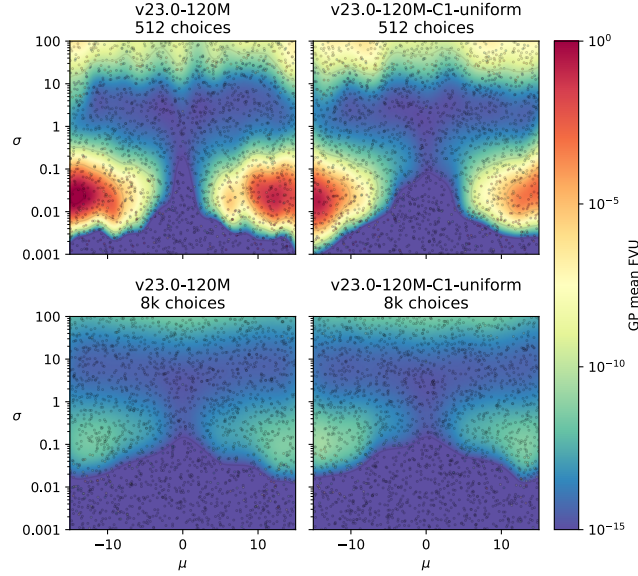


Figure 21. Constants-prior comparison on the Gaussian family. Each panel shows per-instance $\log_{10}$ FVU for 2,048 random (μ, σ) draws (heatmap: Matérn-kernel Gaussian-process mean over $(\mu, \log \sigma)$; dots: raw samples coloured by their own FVU). **Top row:** 512 candidate skeletons. **Bottom row:** 8,192. **Left:** broader $\mathcal{N}(0, 5)$ prior used in this work. **Right:** narrow $\mathcal{U}(1, 5)$ prior of Biggio et al. (2021).

O. SimpliPy Rules

Table 7. Random subset of the simplification rules from the SIMPLIFY engine organized into pattern-replacement pairs. The shown expressions are one-dimensional, hence we use x as the variable token. The constant placeholder $\diamond$ represents an arbitrary finite constant. Infinities are carried through the simplification process to enable simplifications such as $\exp(-\frac{1}{x-x}) \xrightarrow{\text{Cancel}} \exp(-\frac{1}{0}) \xrightarrow{\text{Rule 108}} \exp(-\infty) \xrightarrow{\text{Rule 37}} 0$.

PATTERN	→	REPLACEMENT	PATTERN	→	REPLACEMENT
$(\frac{-\infty}{4})^{\sqrt[3]{\pi}}$	→	∞	$\frac{4x}{\pi}$	→	$\diamond \cdot x$
$(-\infty)^x \cdot (e \cdot x)$	→	$(-\infty)^x$	$(x \cdot \infty) + \frac{x}{\text{NaN}}$	→	NaN
$\frac{1/\pi}{0/(-\infty)}$	→	∞	$\frac{x+(-\infty)}{\sinh(\pi)}$	→	$-\infty$
$(\infty \cdot 1) \cdot \frac{\pi}{-1}$	→	$-\infty$	$\frac{e}{-1} \cdot (-\infty)^\infty$	→	$-\infty$
$\sinh(\pi)^{x-\infty}$	→	0	$(x \cdot 0)^{-1/e}$	→	∞
$\arcsin(0)^{\sqrt[3]{\pi}}$	→	0	$(\pi \cdot \pi) \cdot 2(-\infty)$	→	$-\infty$
$(-1 - \pi) \cdot (\infty \cdot \pi)$	→	$-\infty$	$\sinh(-\infty)^{\arctan(e)}$	→	∞
$\frac{x+\infty}{5(-1)}$	→	$-\infty$	$(1/2)^{x/3}$	→	$\diamond^x$
$\frac{\infty^\pi}{\pi+(-1)}$	→	∞	$\arcsin(-1) \cdot \infty^4$	→	$-\infty$
$\frac{\sqrt{-\infty}}{4\pi}$	→	∞	$(\pi^\pi)^{\infty^3}$	→	∞

P. Memory Requirements

The pre-discovered rewrite table comes at a fixed memory cost. SIMPLIFY occupies ~ 412 MB of resident memory versus ~ 58 MB for SYMPY⁴, a delta of ~ 350 MB that does not grow with the size of the input expression and represents a small fraction of the total memory budget of a training run (≥ 16 GB VRAM, CPU resident sets of the data loader, the transformer model, and per-worker simulation state). We observe no simplification-induced memory growth during training or inference, including for expressions with up to $D = 17$ input variables and $|\tau| \leq 35$ symbols (Appendix K).

For amortized SR the binding constraint is throughput rather than memory. At 100M expressions per training run with 8-thread parallel simplification (8T, matching our cluster training setup), ~ 1.7 ms per expression with SIMPLIFY versus ~ 210 ms with SYMPY (mean, Section 6.2) translates to ~ 6 hours versus ~ 30 days of wall-clock simplification.

⁴Python 3.11, $L_{\max} = 7$ rule set, RSS measured immediately after the engine class is loaded and the rule table is mapped into memory.

Q. Per-Expression Results

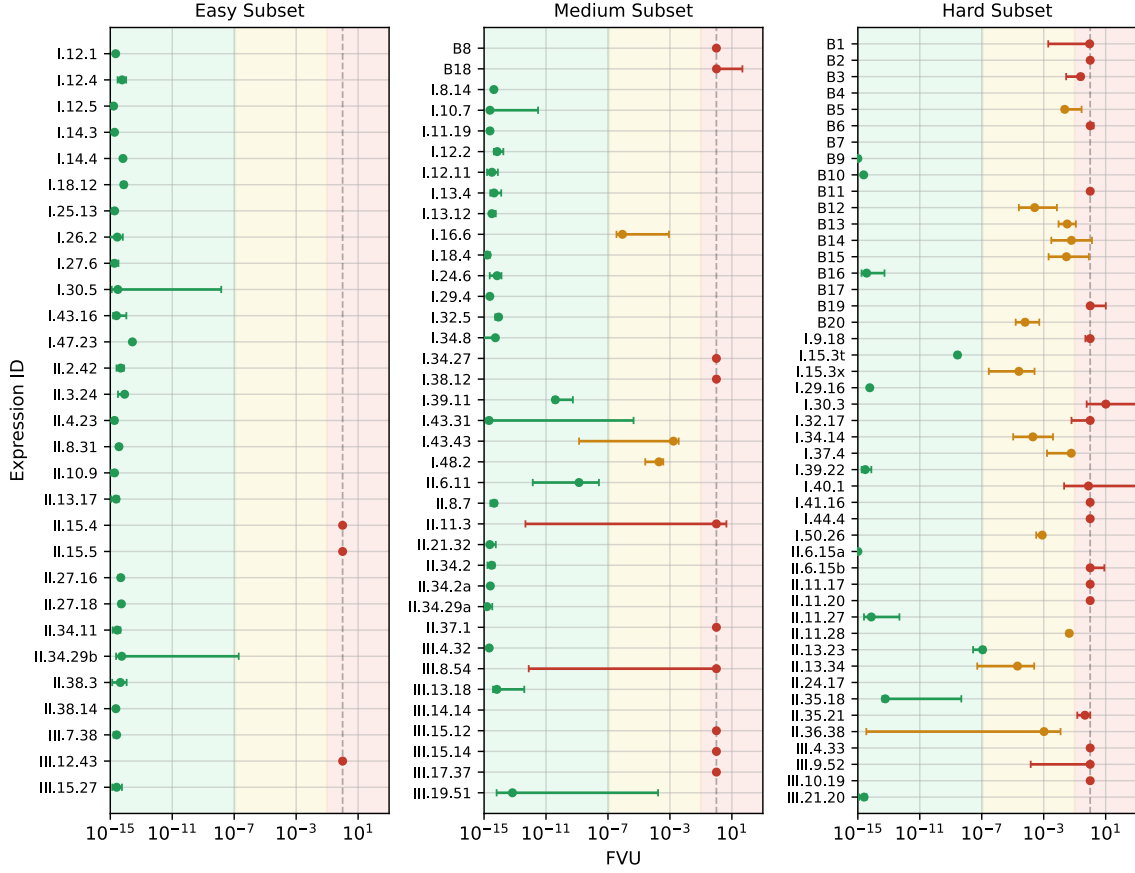


Figure 22. Median and 95% confidence intervals of the FVU with v23.0-120M, $\gamma = 0.05$, 8 optimizer restarts, 256k samples $\approx$ 1000s across individual expressions in the FASTSRB benchmark stratified by difficulty level following (Matsubara et al., 2024). All but three “easy” expressions are solved perfectly (left, green dots). In both “medium” and “hard” categories, the share of approximate fits (orange) and failures (red, clustered around the gray dashed line at $FVU \approx 1$, i.e. no better than the mean predictor) grows with difficulty.

R. Token Embeddings

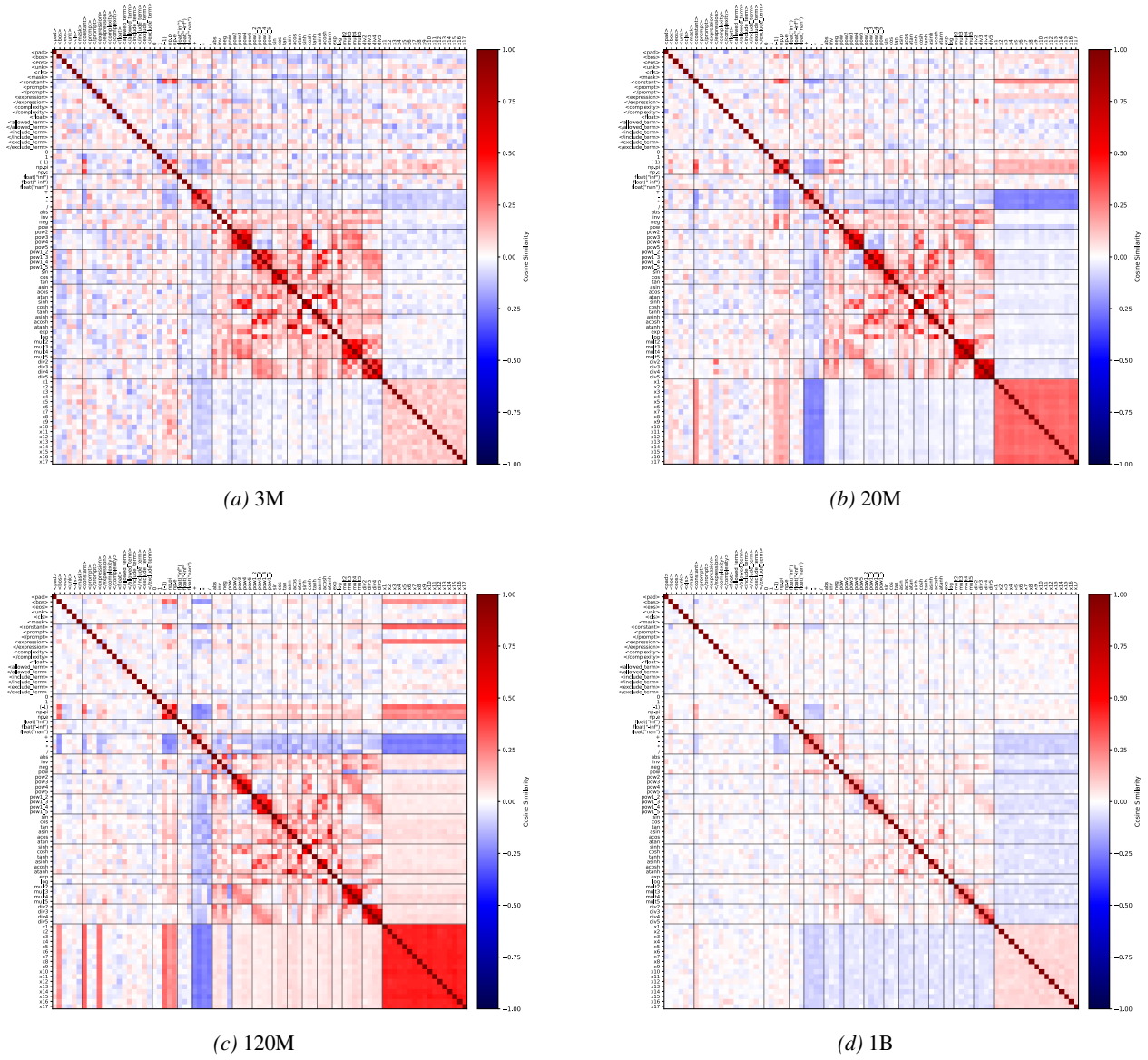


Figure 23. Pairwise token embedding similarity for the Flash-ANSR model family (v23.0) across model sizes. Across all scales, we observe high similarity of semantically related operators (e.g., binary operators, trigonometric functions, exponentials and powers) and variable tokens, and low similarity between different types of tokens (e.g., operators vs. variables). The overall strength of these relationships varies across model sizes with no discernible pattern.